UNIVERSITI TUN HUSSEIN ONN MALAYSIA

BIE20404: VISUAL PROGRAMMING

SEMESTER 2, SESSION 2025/2026

PROJECT TITLE: TIC-TAC-TOE

GROUP 13

LECTURER: Dr. Mohd Zanes Bin Sahid

NO.	STUDENT'S NAME	MATRIX NO.
1.	MUHAMMAD FAIZ IKHWAN BIN KAMARUDIN	AI240014
2.	MOHAMAD IRFAN BIN MOHD KAMILI	AI240015
3.	MUHAMMAD AMIRUL HAFIZ BIN JASMANI	AI240040
4.	MUHAMMAD FARHAN BIN MAD.ZIN	AI240053

TABLE OF CONTENTS

1.0 INTRODUCTION 3
2.0 PROBLEM BACKGROUND 4
2.1 Limited Customization in Traditional Tic-Tac-Toe 4
2.2 Lack of Different Gameplay Modes 4
2.3 Lack of Persistent User Settings 4
2.4 Need for Responsive and Reliable Desktop Applications 4
3.0 PROJECT OBJECTIVES 5
3.1 Develop a Java Swing Graphical User Interface 5
3.2 Implement Singleplayer and Multiplayer Modes 5
3.3 Support Dynamic Board Sizes from 3x3 to 6x6 5
3.4 Implement Database-Based Settings Persistence 5
3.5 Apply Multithreading and Exception Handling 5
4.0 METHODOLOGY 6
4.1 Requirement Analysis 6
4.2 System Design 6
4.3 Implementation 6
4.4 Testing 6
4.5 Documentation 6
5.0 ANALYSIS AND DESIGN 7
5.1 System Analysis 7
5.2 System Design 7
5.2.1 Overall Class Diagram 7
5.2.2 Main Class 9
5.2.3 WelcomePanel Class 10
5.2.4 GamePanel Class 11
5.2.5 GameBoard Class 12
5.2.6 GameModel Class 13
5.2.7 GameController Class 14
5.2.8 GameMode Interface 15
5.2.9 SinglePlayerMode Class 16
5.2.10 MultiPlayerMode Class 17
5.2.11 BotAI Interface 18
5.2.12 RandomBot Class 19
5.2.13 SmartBot Class 20
5.2.14 MusicPlayer Class 21
5.2.15 DatabaseConnection Class 22
5.2.16 SettingsDAO Class 23
5.2.17 GameException Class 24
5.3 System Functions and Java Implementation 25
5.3.1 Welcome Screen 25
5.3.2 Gameplay Screen 25
5.3.3 Singleplayer Mode 25

5.3.4 Multiplayer Mode 25
5.3.5 Dynamic Board Generation 25
5.3.6 Bot Decision-Making Algorithm 25
5.3.7 Win and Draw Detection 25
5.3.8 Settings Management 26
5.3.9 Audio Management 26
5.3.10 Database Operations 26
5.4 Java Libraries and Programming Concepts 26
6.0 IMPLEMENTATION 27
6.1 Object-Oriented Programming Concepts 27
6.1.1 Encapsulation 27
6.1.2 Inheritance 27
6.1.3 Abstraction 27
6.1.4 Polymorphism 27
6.1.5 Composition 27
6.1.6 Interface Implementation 28
6.2 Event-Driven Programming 28
6.3 Multithreading Implementation 30
6.3.1 Match Timer Thread 30
6.3.2 Bot Calculation Thread 30
6.3.3 Background Music Thread 31
6.4 Exception Handling 33
6.4.1 Database Exceptions 33
6.4.2 Audio Exceptions 33
6.4.3 Interface and Game Logic Exceptions 33
6.5 JDBC Database Connectivity 34
6.6 Screenshot of Output: Welcome Screen 36
6.7 Screenshot of Output: Gameplay Screen 37
6.7.1 Standard 3x3 Board 37
6.7.2 Custom 4x4, 5x5, and 6x6 Boards 38
6.7.3 Singleplayer Gameplay 39
6.7.4 Multiplayer Gameplay 39
6.7.5 Winning-Line Highlight 40
6.7.6 Draw Result 41
6.7.7 Mute and Unmute Function 42
6.8 Screenshot of Output: Settings Screen 43
6.9 Screenshot of Output: MySQL Database 44
7.0 SYSTEM TESTING 45
7.1 Welcome-Screen Navigation Test 45
7.2 Valid and Invalid Move Test 45
7.3 Singleplayer Bot Test 45
7.4 Multiplayer Turn Test 45
7.5 Win Detection Test 45
7.6 Draw Detection Test 45
7.7 Board-Size Test 46
7.8 Audio Test 46

7.9 Timer Test 46
7.10 Database Save and Load Test 46
7.11 Exception-Handling Test 46
8.0 RESULTS AND CONCLUSION 47
9.0 FUTURE IMPROVEMENTS 48
10.0 REFERENCES 49
APPENDICES 50
Appendix A: Database SQL Script 50
Appendix B: Important Source-Code Extracts 51
Appendix C: Complete Test-Case Table 57

1.0 INTRODUCTION

This project develops a desktop Tic-Tac-Toe application using the Java Swing graphical user interface framework. The submitted program provides a welcome screen, a gameplay screen, a settings screen, singleplayer and multiplayer modes, selectable board sizes from 3x3 to 6x6, match information, background music controls, database-based settings persistence, multithreading, and exception handling. The application is organized into user-interface, game-logic, game-mode, bot, database, and exception packages.

The standard game rule is implemented by allowing two symbols, X and O, to occupy empty cells in alternating turns. A match ends when every cell in one complete row, column, main diagonal, or anti-diagonal contains the same non-empty symbol. On custom boards, the submitted logic requires a full line equal to the selected board dimension. A draw occurs when every cell is occupied, and no winning line exists. This functionality is executed using GameModel.java and is synchronized with the GUI using GameController.java.

The processing of events from Swing takes place within the Event Dispatch Thread, while tasks that take time or run constantly are delegated to background threads. As per Oracle documentation, most Swing operations should be carried out on the Event Dispatch Thread, and worker threads are best suited for background processing operations.

2.0 PROBLEM BACKGROUND

2.1 Limited Customisation in Traditional Tic-Tac-Toe

A fixed 3x3 board provides only one level of game complexity. The limitation of a constant 3x3 board can be only one difficulty in the game. The requirements for the project state that there is a default 3x3 board as well as the possibility of creating custom NxN boards up to 6x6 size. That is why the SettingsPanel offered by the applicant allows users to choose from 3x3, 4x4, 5x5, and 6x6 sizes. At the same time, GameBoard creates the needed number of JButton fields on the basis of GridLayout.

2.2 Lack of Different Gameplay Modes

A constant opponent type does not provide the opportunity to play both alone and with a second local player. The solution was found in introducing the interface of GameMode, which has two realizations - SinglePlayerMode and MultiPlayerMode.

2.3 Lack of Persistent User Settings

Without persistence, players will need to choose their mode, board size, timer visibility, board information, and player counter again every time the game is launched. SettingsDAO.java file includes saving and loading operations for the game_settings table, while Main.java tries to load the values before displaying the GUI. In the case of unavailability of the database, the application will keep in-memory default values and show a proper warning.

2.4 Need for Responsive and Reliable Desktop Applications

A desktop application may become unresponsive when some lengthy or continuous tasks are done right in the event handler. In the presented implementation, the match timer, delay and calculation of the bot, and music are implemented as separate threads. Synchronized methods of the GameModel class and exceptions (GameException with severity level) are used here as well.

3.0 PROJECT OBJECTIVES

3.1 Develop a Java Swing Graphical User Interface

To implement a non resizable desktop window, with CardLayout switching between WelcomePanel, GamePanel and SettingsPanel, with the use of Swing components andcustom-painted icons.

3.2 Implement Singleplayer and Multiplayer Modes

To support Human v.s. Bot play and Human v.s. Human alternating games via GameMode abstraction.

3.3 Support Dynamic Board Sizes from 3x3 to 6x6

To generate required to create the board dynamically and perform full row, column and diagonal win detection based on specified size.

3.4 Implement Database-Based Settings Persistence

Saving and loading game mode, board size and switches for match information via JDBC, MySQL, PreparedStatement and ResultSet.

3.5 Apply Multithreading and Exception Handling

Maintaining timer, bot and sound operations independently of interface events, and handling possible database, sound, index and interface errors without uncontrolled application shutdown.

4.0 METHODOLOGY

The project takes an iterative approach where requirement identification, system partitioning into collaborating classes, implementation by package, and verification of the source through compilation, interface capture, static analysis, and automated logic tests take place.

4.1 Requirement Analysis

The progress report and interface requirement specifications were reviewed to determine screens, controls, game rules, settings, audio, timer, database, exceptions, and threads requirements. Finally, the requirements were cross-referenced with the submitted Java and SQL files in the final report, as opposed to taking it for granted that all intended features have been implemented.

4.2 System Design

The system was partitioned into Main frame, panel visualizers, board, model, controller, mode interface, bot interface, persistence, and exception classes. The UML diagrams in Section 5.2 show the actual names of classes and relationships in the submitted Java code.

4.3 Implementation

The system was implemented in Java SE 21 using Swing and AWT for the interface, Java Sound for WAV playback, java.sql for JDBC operations, and java.lang.Thread for concurrent work. The source structure contains 18 Java files and one SQL schema file.

4.4 Testing

The source file is compiled by javac 21. Tests included move legality, win conditions on board sizes 3 to 6, draws, bot wins, bot blocks, random moves, and mode switch tests. The user interface screen shots were taken from a compiled version of the submitted user interface. Database operation could not be confirmed due to lack of connector JAR and database server.

4.5 Documentation

The report records validated design, responsibilities of classes, implementation strategies, output screen prints, test documentation, limitations, references, SQL script, and code excerpts. Unproven statements made in the sample report, like having three selectable difficulties for the bot or a Minimax algorithm, are not reiterated since they are not part of the submission.

5.0 ANALYSIS AND DESIGN

5.1 System Analysis

The suggested application will be event-based desktop game. The user starts in WelcomePanel and makes either Play or Settings choice. Play will lead him to the GamePanel. Settings leads him to SettingsPanel, in which he makes his choice between Singleplayer or Multiplayer, sets his board size from 3x3 up to 6x6 and toggles three options about match information. Save & Back method updates GameModel, tries to save its values using SettingsDAO, builds game board anew and goes back to the welcome screen.

During the game, GameController receives button events from GameBoard. Then it asks GameModel whether it should validate the move and store it, updates symbol and counters, determines whether it is a victory or draw condition and calls for the next turn using current GameMode value. If it is Singleplayer, GameController locks the board, launches a daemon thread with the bot, gets a move from SmartBot and puts it into the board on Event Dispatch Thread. First, SmartBot looks for an immediately winnable cell, then for an immediate human threat and, finally, passes the move to RandomBot.

game _settings and game_stats tables exist in the database schema design. Java persistence layer at the moment can write/read game_settings only. Table for game_stats is created and populated in schema.sql, but there is no DAO update for it among the submitted ones. Also class DatabaseException.java exists but is not used, while the active database classes are using GameException class.

5.2 System Design

5.2.1 Overall Class Diagram

Figure 5.2.1 shows the proven relationship between the classes provided. The solid arrows represent containment, control, inheritance, or interfaces being implemented, while the dashed arrows denote usage or exception throwing. SettingsPanel and DatabaseException are part of the diagram because they exist in the source code, even though they are not explicitly mentioned in the class subsection request.

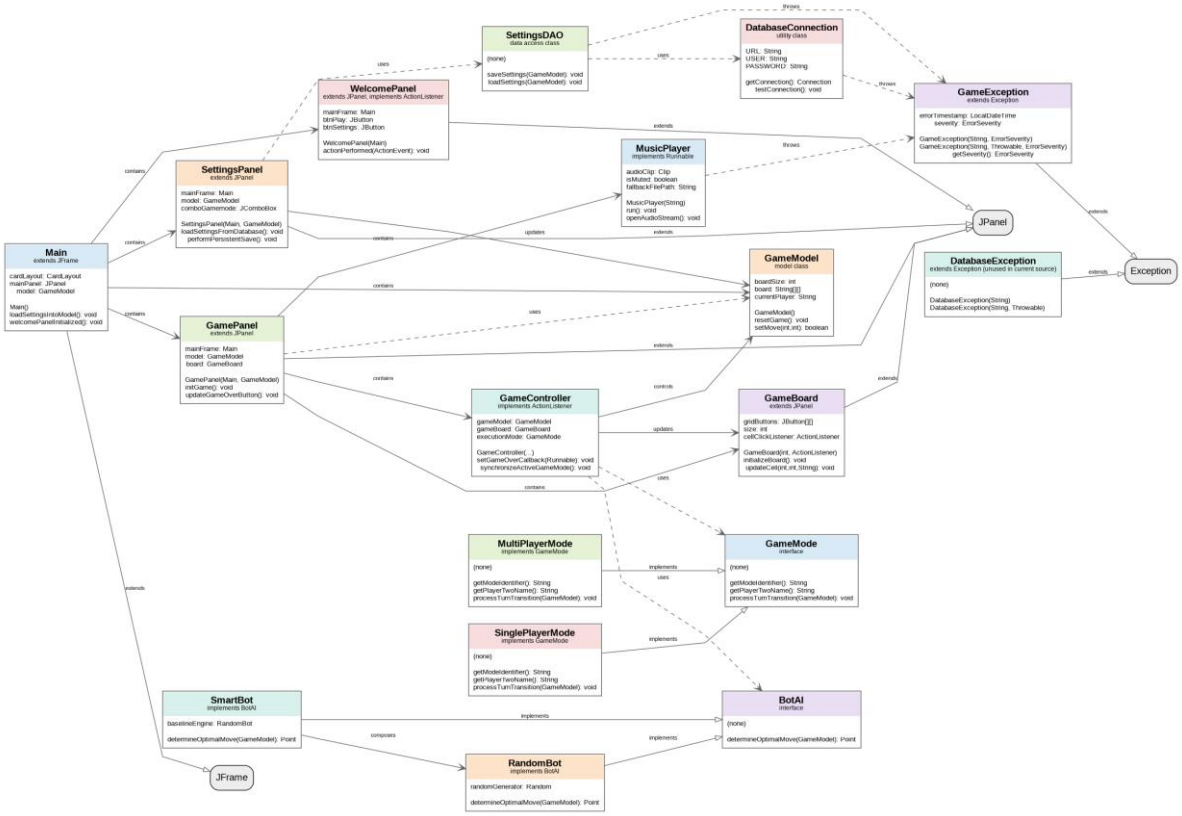
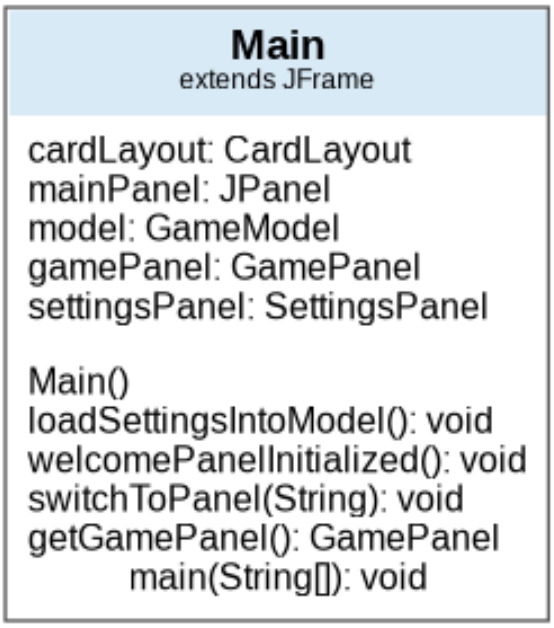


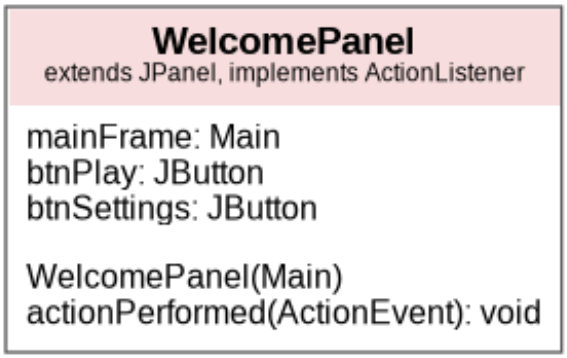
Figure 5.2.1: Overall class diagram derived from the submitted Java source.

5.2.2 Main Class

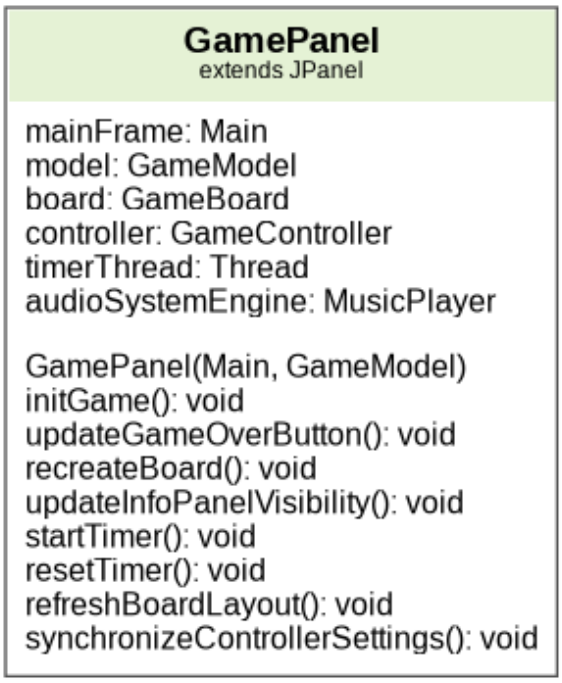
Java Class	Class Diagram
Main	 *Figure 5.2.2: UML Diagram of Main.*

Description	Main.java is the frame of the application and entry point for the program. The class extends JFrame and creates a non-resizable frame of 520 x 680 pixels using CardLayout and the following panels: WelcomePanel, GamePanel, and SettingsPanel. A common GameModel object is initialized before the creation of the panel, making sure all panels will work in the same runtime environment with the same score. loadSettingsIntoModel() calls SettingsDAO.loadSettings(); GameException is thrown to make sure the program can still use default values even if MySQL cannot be accessed. Main() method also initializes the database connection in separate method DatabaseConnection.testConnection() and outputs the error message if MySQL cannot be accessed. The frame is created through SwingUtilities.invokeLater(), making sure all swing components will be created in EDT.

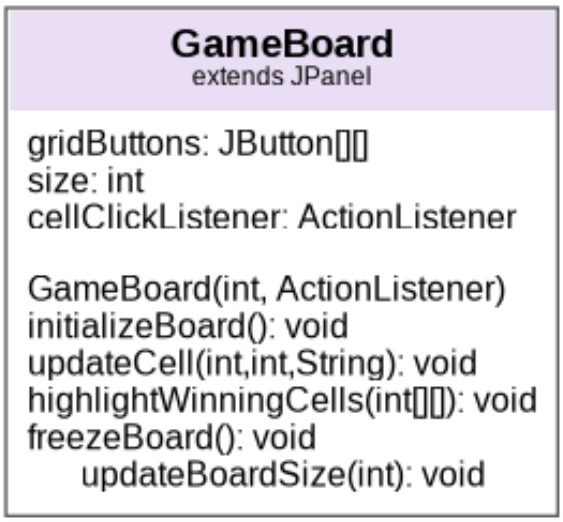
5.2.3 WelcomePanel Class

Java Class	Class Diagram
WelcomePanel	 *Figure 5.2.3: UML Diagram of WelcomePanel.*
Description	The WelcomePanel.java is the first screen displayed on the application. This screen inherits from JPanel and implements the ActionListener interface. GridBagLayout places the title in the centre, while a two-row GridLayout displays the Play and Settings buttons. The Play button is attached to the same panel, such that the actionPerformed() method will change the card in CardLayout to GamePanel. The Settings button, on the other hand, has its own ActionListener which navigates to SettingsPanel.

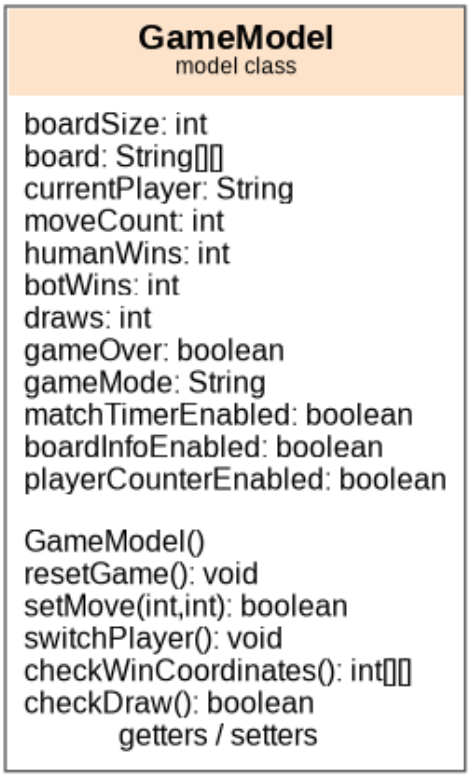
5.2.4 GamePanel Class

Java Class	Class Diagram
GamePanel	 *Figure 5.2.4: UML Diagram of GamePanel.*
Description	GamePanel.java creates the GUI for the game. It has the status label, cards displaying information about the game, GameBoard, Play/Again Try/Again button, Menu button, mute button, timer state, GameController, and MusicPlayer. The method initGame() generates a board of size GameModel.getBoardSize() and associates it with a controller. Method recreateBoard() destroys the existing board and rebuilds it based on the modified settings. Method updateInfoPanelVisibility() modifies all three settings switches. Timer thread increases the value of secondsElapsed by one each second and updates the label using SwingUtilities.invokeLater(). The constructor starts MusicPlayer in a daemon thread.

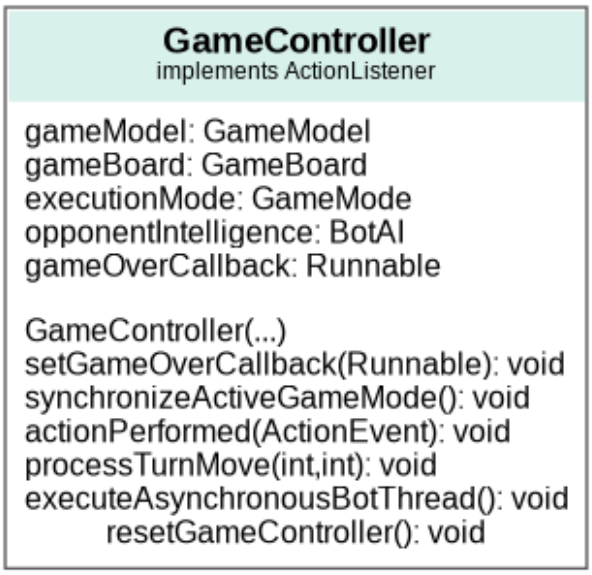
5.2.5 GameBoard Class

Java Class	Class Diagram
GameBoard	 *Figure 5.2.5: UML Diagram of GameBoard.*
Description	GameBoard.java extends the reusable JPanel class and draws the NxN board. The method initializeBoard() instantiates size * size number of JButtons, sets their row and column values as client properties, and adds the passed ActionListener to it. The methods updateCell() shows X in blue and O in coral color. highlightWinningCells() changes the background color to teal along with changing the color of all coordinates returned by GameModel to white color. freezeBoard() freezes all cells until the bot finishes thinking, and updateBoardSize() rebuilds the grid with new size.

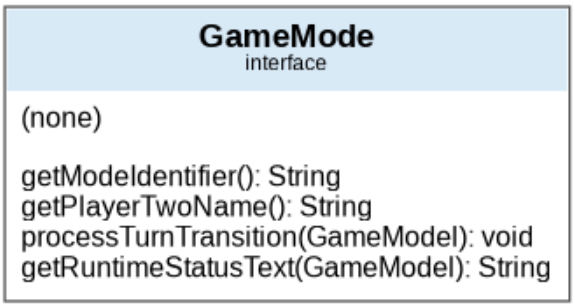
5.2.6 GameModel Class

Java Class	Class Diagram
GameModel	 *Figure 5.2.6: UML Diagram of GameModel.*
Description	All mutable data for the game and the settings is stored in GameModel.java file. The private variables include the board, its dimension, current symbol, number of moves, scores, draw count, game over flag, mode, player symbols, and flags for matching information. Most of the game state methods are synchronized since they can be accessed by both the Event Dispatch Thread and bot thread. The method setMove() accepts only valid moves which are in range, not occupied and not done after the end of the game. The method checkWinCoordinates() checks for full rows, columns, main diagonal, and anti-diagonal, returning exact coordinates for highlighting. The method checkDraw() needs a full board without any winning line.

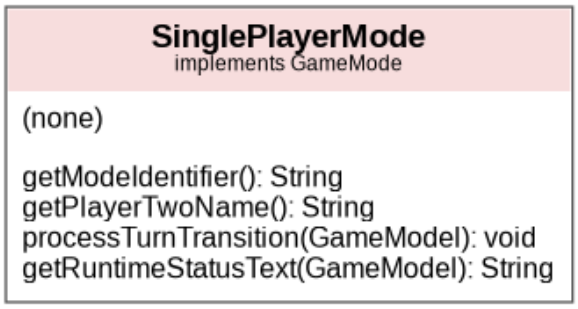
5.2.7 GameController Class

Java Class	Class Diagram
GameController	 *Figure 5.2.7: UML Diagram of GameController.*
Description	The GameController.java manages the updates of the model and view. It implements the interface ActionListener, hence all clicks in a cell invoke the method actionPerformed(). The method synchronizeActiveGameMode() determines either SinglePlayerMode or MultiPlayerMode according to the model. The processTurnMove() logs the move, modifies the state of the cell and the counters, checks for a win or tie game, updates scores and the state, and calls the end of game callback function. In singleplayer mode, the executeAsynchronousBotThread() thread sleeps for 750 milliseconds, requests the move from SmartBot, and gets back to Event Dispatch Thread to build the board.

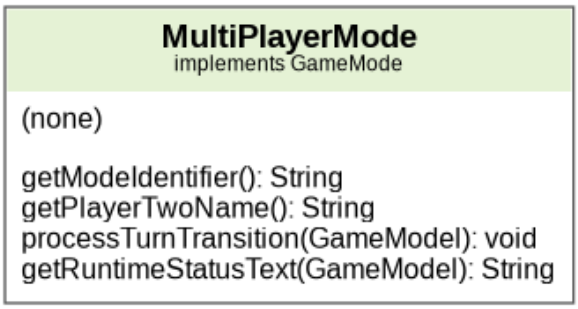
5.2.8 GameMode Interface

Java Class	Class Diagram
GameMode	 *Figure 5.2.8: UML Diagram of GameMode.*
Description	The GameMode.java file is an interface that specifies behavior depending on the game mode, namely the mode identifier, player two name, change of turns, and status string. The actual implementation is contained in the GameController class, which maintains an instance of the abstract type rather than the concrete one.

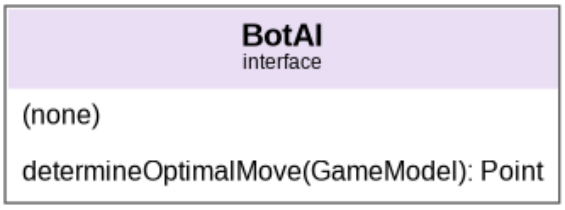
5.2.9 SinglePlayerMode Class

Java Class	Class Diagram
SinglePlayerMode	 *Figure 5.2.9: UML Diagram of SinglePlayerMode.*
Description	The SinglePlayerMode.java class is an implementation of the GameMode for human versus bot games. It determines that the current game mode is Singleplayer, sets the name of the second player as Bot, changes the active mark when there is a successful turn, and returns either "Your Turn" or "Bot thinking...", depending on GameModel.getCurrentPlayer().

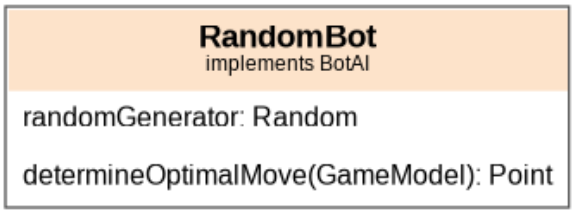
5.2.10 MultiPlayerMode Class

Java Class	Class Diagram
MultiPlayerMode	 *Figure 5.2.10: UML Diagram of MultiPlayerMode.*
Description	MultiPlayerMode.java is an implementation of GameMode for two local human players. The class recognizes the type of game mode as Multiplayer, gives the name of the second player as Player 2, switches the current symbol after every move that has been made, and returns "Player 1 Turn" or "Player 2 Turn." This class does not generate a bot thread, as it will be called by GameController when the model game mode is Singleplayer.

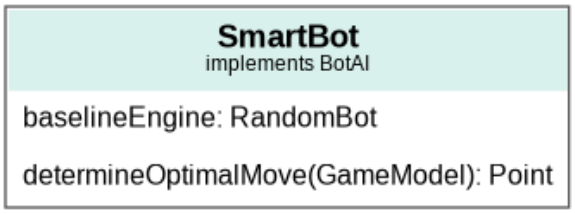
5.2.11 BotAI Interface

Java Class	Class Diagram
BotAI	 *Figure 5.2.11: UML Diagram of BotAI.*
Description	BotAI.java is the interface for bot strategy. It provides determineOptimalMove(GameModel) method that returns an object of java.awt.Point with the selected row and column. Both RandomBot and SmartBot classes have implemented this interface. Interface enables GameController class to rely on the abstract bot while strategies can be easily replaced.

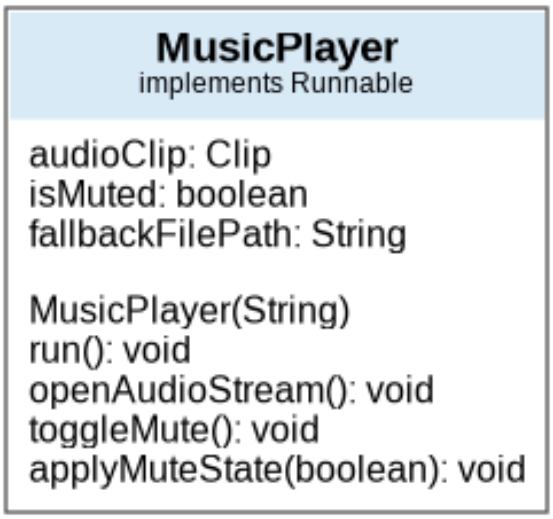
5.2.12 RandomBot Class

Java Class	Class Diagram
RandomBot	 *Figure 5.2.12: UML Diagram of RandomBot.*
Description	RandomBot.java implements BotAI by looking at the current board, gathering all the empty coordinates, and choosing one randomly using java.util.Random. It can only return null when there is no possible move left. This algorithm is used directly for automatic testing and indirectly as the fallback for SmartBot.

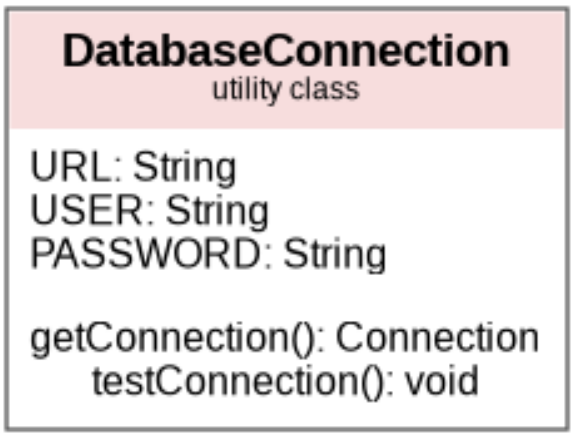
5.2.13 SmartBot Class

Java Class	Class Diagram
SmartBot	 *Figure 5.2.13: UML Diagram of SmartBot.*
Description	The SmartBot.java is the implementation of BotAI using a heuristic strategy involving three layers. The first layer involves the temporary placement of the bot symbol in all empty cells checking if such a move results in a winning move. The second layer involves placing the human symbol temporarily to identify any human winning move which is then blocked. Otherwise, the task is delegated to the underlying RandomBot object. This class does not involve the use of the Minimax algorithm and there is no difficulty selector interface implemented in the class.

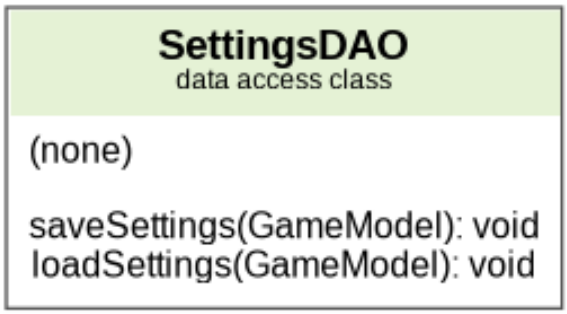
5.2.14 MusicPlayer Class

Java Class	Class Diagram
MusicPlayer	 *Figure 5.2.14: UML Diagram of MusicPlayer.*
Description	The MusicPlayer.java file implements Runnable so that audio loading and looping can be done in the background. First, the program checks whether there is a valid file path provided before checking for the classpath resource. A valid stream is created from the Clip and then it loops infinitely. The method toggleMute uses BooleanControl.Type.MUTE when it is available; otherwise, it uses MASTER_GAIN. Any unsupported format, unavailable audio lines, and I/O exceptions are caught by a GameException which will be thrown while the application runs without any error..

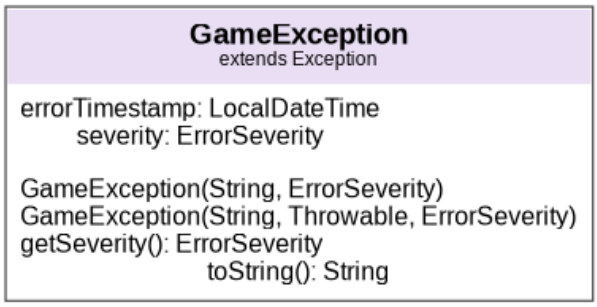
5.2.15 DatabaseConnection Class

Java Class	Class Diagram
DatabaseConnection	 *Figure 5.2.15: UML Diagram of DatabaseConnection.*
Description	DatabaseConnection.java consolidates the JDBC URL, user ID, and password. getConnection() method loads driver com.mysql.cj.jdbc.Driver and invokes DriverManager.getConnection() for tic_tac_toe_db located at localhost port 3306. Failure of loading driver and SQL related exceptions are converted to GameException with severity values. testConnection() method utilizes try-with-resources construct to close Connection and throws exception based on whether the connection was closed or broken. Source can be compiled without connector but needs external Connector/J jar mentioned in Eclipse classpath file at runtime.

5.2.16 SettingsDAO Class

Java Class	Class Diagram
SettingsDAO	 *Figure 5.2.16: UML Diagram of SettingsDAO.*
Description	SettingsDAO.java is the static data-access class for the game_settings table. The method saveSettings() will perform an UPDATE statement on row id 1 using parameters or create the row when no UPDATE happens. Method loadSettings() performs SELECT statement using parameters and loads values of mode, size, and three flags to the GameModel. The try-with-resources statement is used for all Connection, PreparedStatement, ResultSet instances. SQL exceptions are caught by GameException. The provided code doesn't use game_stats so score storing is not available yet.

5.2.17 GameException Class

Java Class	Class Diagram
GameException	 *Figure 5.2.17: UML Diagram of GameException.*
Description	The GameException is the custom checked exception that is currently in use. This extends Exception, maintains the timestamp of creation, and can have any of the following severity levels - INFO, WARNING, or CRITICAL. The constructors can be used to create GameExceptions with or without a cause along with the messages. The toString() function provides a format for timestamp, severity, and message. DatabaseConnection, SettingsDAO, and MusicPlayer classes make use of GameException class.

5.3 System Functions and Java Implementation

5.3.1 Welcome Screen

The welcome screen contains the title of the application and two buttons. Play calls Main.switchToPanel("GamePanel"). Settings calls Main.switchToPanel("SettingsPanel"). This screen does not involve setting up any games nor displaying any difficulty screens.

5.3.2 Gameplay Screen

The game screen consists of status information, time, moves, two score trackers, an animated grid, menu navigation, replay function, and mute function. The responsibilities of rule handling are given to GameController by GamePanel, while those of cell handling are delegated to GameBoard.

5.3.3 Singleplayer Mode

The human player makes the move X. Following the completion of the human move, the controller freezes the board and initiates the Bot thread. The SmartBot chooses the letter O by using the following logic: win, block, and finally, random moves.

5.3.4 Multiplayer Mode

X belongs to Player 1 and O belongs to Player 2. There is no need for bot thread initialization. MultiPlayerMode takes turn with the current symbol and changes status text accordingly. The score variables remain the same; their identifiers are depicted by human and robot images.

5.3.5 Dynamic Board Generation

The function GameBoard.initializeBoard() uses GridLayout(size, size, 8, 8), creating one button for each row and column. SettingsPanel limits options to 3, 4, 5, and 6. GamePanel reconstructs the board with changed settings.

5.3.6 Bot Decision-Making Algorithm

The SmartBot checks each vacant space for the possibility of a win for the bot, followed by each vacant space for the possibility of a human win that has to be stopped. This is a one-level heuristic and not the recursive Minimax algorithm.

5.3.7 Win and Draw Detection

GameModel evaluates rows and columns that are completed, and then diagonal lines as well. Coordinates are delivered to GameBoard for highlighting the winning line in teal color. In case of draw, moveCount needs to equal the square of boardSize with no line detected.

5.3.8 Settings Management

SettingsPanel includes mode and board combo-boxes, three toggle-switches, Reset to default and Save and Back buttons. During saving, first GameModel is updated and then SettingsDAO.saveSettings() is attempted. Then GamePanel is rebuilt, and the application goes back to WelcomePanel.

5.3.9 Audio Management

GamePanel launches the MusicPlayer as a Daemon thread. MusicPlayer uses AudioSystem, AudioInputStream, Clip, BooleanControl, and FloatControl. Toggle mute is done using a UI button and the icon changes accordingly. The source code is just for background music; there are no other sounds separately for moving or results.

5.3.10 Database Operations

DatabaseConnection establishes connections to MySQL. SettingsDAO performs operations using PreparedStatement, including updates, inserts, and selects for row id 1. The SQL script defines and populates tables named game_settings and game_stats. However, game_settings is the only table that has an access path via Java.

5.4 Java Libraries and Programming Concepts

Library / Package	Verified Usage	Purpose
javax.swing	JFrame, JPanel, JButton, JLabel, JComboBox, JScrollPane, JOptionPane, SwingUtilities	Window construction, controls, dialogs, navigation, and EDT updates.
java.awt	CardLayout, BorderLayout, GridLayout, GridBagLayout, FlowLayout, Graphics2D, Point	Screen switching, component layout, custom drawing, and bot coordinates.
java.awt.event	ActionListener, ActionEvent, MouseAdapter	Button and toggle event handling.
javax.sound.sampled	AudioSystem, AudioInputStream, Clip, controls	Background WAV loading, looping, and mute control.
java.sql	Connection, DriverManager, PreparedStatement, ResultSet, SQLException	JDBC settings persistence.
java.lang	Thread, Runnable, synchronized	Timer, bot, and audio concurrency.
java.util	Random, ArrayList, List	Legal-move collection and random fallback selection.
java.time	LocalDateTime	Exception timestamp recording.

The external MySQL dependency does not exist inside the ZIP file. The .classpath file in the Eclipse IDE makes use of the absolute path on Windows to reference mysql-connector-j-8.0.33.jar. Connector/J is referred to by MySQL as the official JDBC driver for MySQL.

6.0 IMPLEMENTATION

The design is based on a MVC-like architecture without explicitly creating an MVC design. GameModel contains the game state and rules, while GamePanel and GameBoard contain the view. GameController takes care of handling events. The Main class manages navigation between screens, while SettingsDAO and DatabaseConnection make up the persistence tier.

6.1 Object-Oriented Programming Concepts

6.1.1 Encapsulation

The GameModel class holds board information, scores, settings, and turn data in private variables. The public methods check and manipulate the game state. The synchronized methods ensure the security of the shared state.

6.1.2 Inheritance

Main extends JFrame; WelcomePanel, GamePanel, SettingsPanel, and GameBoard extend JPanel; GameException and DatabaseException extend Exception. Every subclass utilises and customises the behaviour provided by the respective superclass.

6.1.3 Abstraction

GameMode and BotAI provide only those operations that are needed by the controller. The controller does not require knowledge about all aspects of SinglePlayerMode, MultiPlayerMode, RandomBot, and SmartBot.

6.1.4 Polymorphism

The executionMode field has type GameMode but stores either SinglePlayerMode or MultiPlayerMode. Bot strategies share the BotAI type, and SmartBot delegates to a RandomBot that implements the same method contract.

6.1.5 Composition

Main owns shared panels and GameModel. GamePanel owns GameBoard, GameController, and MusicPlayer. SmartBot owns a RandomBot fallback. These contained objects collaborate to implement complete behaviour.

6.1.6 Interface Implementation

WelcomePanel and GameController implement ActionListener, MusicPlayer implements Runnable, SinglePlayerMode and MultiPlayerMode implement GameMode, and RandomBot and SmartBot implement BotAI.

```
        this.boardSize = 3; // Default visual grid setting
        this.gameMode = "Singleplayer";
        this.player1Symbol = "X";
        this.player2Symbol = "O";
        resetGame();
    }

    public synchronized void resetGame() {
        this.board = new String[boardSize][boardSize];
        for (int i = 0; i < boardSize; i++) {
            for (int j = 0; j < boardSize; j++) {
                board[i][j] = "";
            }
        }
        this.currentPlayer = player1Symbol;
        this.moveCount = 0;
        this.gameOver = false;
    }

    public synchronized boolean setMove(int row, int col) {
        if (row < 0 || row >= boardSize || col < 0 || col >= boardSize || !board[row][col].equals("") || gameOver) {
            return false;
        }
        board[row][col] = currentPlayer;
        moveCount++;
        return true;
    }

    public synchronized void switchPlayer() {
        if (currentPlayer.equals(player1Symbol)) {
            currentPlayer = player2Symbol;
        } else {
            currentPlayer = player1Symbol;
        }
    }

    // Dynamic NxN Win Detection Engine returning exact grid coordinates for Teal highlighting
    public synchronized int[][] checkWinCoordinates() {
        // 1. Check Rows
        for (int r = 0; r < boardSize; r++) {
            boolean win = true;
            String symbol = board[r][0];
            if (!symbol.equals("")) {
                for (int c = 1; c < boardSize; c++) {
                    if (!board[r][c].equals(symbol)) {
                        win = false;
                        break;
                    }
                }
                if (win) {
                    int[][] coords = new int[boardSize][2];
                    for (int c = 0; c < boardSize; c++) {
                        coords[c][0] = r;
                        coords[c][1] = c;
                    }
                    return coords;
                }
            }
        }

        // 2. Check Columns
        for (int c = 0; c < boardSize; c++) {
            boolean win = true;
            String symbol = board[0][c];
            if (!symbol.equals("")) {
                for (int r = 1; r < boardSize; r++) {
                    if (!board[r][c].equals(symbol)) {
                        win = false;
                        break;
                    }
                }
                if (win) {
                    int[][] coords = new int[boardSize][2];
                    for (int r = 0; r < boardSize; r++) {
                        coords[r][0] = r;
                        coords[r][1] = c;
                    }
                    return coords;
                }
            }
        }

        // 3. Check Primary Diagonal (Top-Left to Bottom-Right)
        String diag1Symbol = board[0][0];
        if (!diag1Symbol.equals("")) {
            boolean win = true;
            for (int i = 1; i < boardSize; i++) {
                if (!board[i][i].equals(diag1Symbol)) {
                    win = false;
                    break;
                }
            }
            if (win) {
                int[][] coords = new int[boardSize][2];
                for (int i = 0; i < boardSize; i++) {
                    coords[i][0] = i;
                    coords[i][1] = i;
                }
                return coords;
            }
        }

        // 4. Check Secondary Diagonal (Top-Right to Bottom-Left)
        String diag2Symbol = board[0][boardSize - 1];
        if (!diag2Symbol.equals("")) {
            boolean win = true;
            for (int i = 1; i < boardSize; i++) {
                if (!board[i][boardSize - 1 - i].equals(diag2Symbol)) {
                    win = false;
                    break;
                }
            }
            if (win) {
                int[][] coords = new int[boardSize][2];
                for (int i = 0; i < boardSize; i++) {
                    coords[i][0] = i;
                    coords[i][1] = boardSize - 1 - i;
                }
```

Figure 6.1: Extract from GameModel.java showing encapsulated state and synchronized game logic.

6.2 Event-Driven Programming

The GUI is driven by events generated by users. Button listeners are created in the WelcomePanel class for navigation purposes. Every GameBoard button keeps track of its own row and column numbers as client properties and sends the ActionEvent object to the GameController. The SettingsPanel class uses action listeners for the reset and save actions and mouse listeners for the custom toggle switch functionality. Lambda functions are used for the replay, menu, and mute buttons in GamePanel.

```java
package main;

import javax.swing.*;
import java.awt.*;
import main.ui.WelcomePanel;
import main.ui.GamePanel;
import main.ui.SettingsPanel;
import main.game.GameModel;
import main.database.DatabaseConnection;
import main.database.SettingsDAO;
import main.exceptions.GameException;

public class Main extends JFrame {
    private CardLayout cardLayout;
    private JPanel mainPanel;
    private GameModel model;
    private GamePanel gamePanel;
    private SettingsPanel settingsPanel;

    public Main() {
        setTitle("Tic-Tac-Toe");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(520, 680);
        setLocationRelativeTo(null);
        setResizable(false);

        cardLayout = new CardLayout();
        mainPanel = new JPanel(cardLayout);
        mainPanel.setBackground(new Color(245, 245, 245));

        model = new GameModel();
        loadSettingsIntoModel();   // Load saved settings before building the UI

        welcomePanelInitialized();

        add(mainPanel);
        cardLayout.show(mainPanel, "WelcomePanel");
    }

    // ------ Load settings from database and apply to model (fixes bugs 2, 4, 5) ------
    private void loadSettingsIntoModel() {
        try {
            // This static method reads the database and updates the given model directly.
            SettingsDAO.loadSettings(model);
        } catch (GameException e) {
            // If database is offline or settings not yet saved, keep defaults (3x3, Singleplayer, all visible)
            System.err.println("Could not load settings: " + e.getMessage());
            // Optionally log the stack trace for debugging
            // e.printStackTrace();
        }
    }

    private void welcomePanelInitialized() {
        try {
            WelcomePanel welcomePanel = new WelcomePanel(this);
            gamePanel = new GamePanel(this, model);
            settingsPanel = new SettingsPanel(this, model);

            mainPanel.add(welcomePanel, "WelcomePanel");
            mainPanel.add(gamePanel, "GamePanel");
            mainPanel.add(settingsPanel, "SettingsPanel");
        } catch (NullPointerException e) {
            System.err.println("Fatal Interface Generation Breakdown: Render construction stack collapsed: " + e.getMessage());
            JOptionPane.showMessageDialog(null,
                "A critical interface render framework exception occurred. Shutting down.",
                "Fatal System Failure",
                JOptionPane.ERROR_MESSAGE);
            System.exit(1);
        }
    }

    public void switchToPanel(String panelName) {
        try {
            cardLayout.show(mainPanel, panelName);
        } catch (IllegalArgumentException e) {
            System.err.println("Attempted invalid CardLayout component redirection to tag identity: " + panelName);
        }
    }

    public GamePanel getGamePanel() {
        return gamePanel;
    }

    public static void main(String[] args) {
        try {
            DatabaseConnection.testConnection();
        } catch (GameException e) {
            System.err.println(e.toString());
            JOptionPane.showMessageDialog(null,
                "MySQL service appears offline on local environment.\nApplication features will operate in safe local memory state.",
                "Database Environment Offline",
                JOptionPane.WARNING_MESSAGE);
        }

        SwingUtilities.invokeLater(() -> {
            try {
                Main frame = new Main();
                frame.setVisible(true);
            } catch (Exception e) {
                System.err.println("Fatal structural failure inside Event Dispatch Thread instantiation frame loop: " + e.getMessage());
            }
        });
    }
```

Figure 6.2: Main.java screen construction and CardLayout navigation.

6.3 Multithreading Implementation

6.3.1 Match Timer Thread

GamePanel.startTimer() creates a thread which pauses itself for 1000 milliseconds, increments secondsElapsed in the time the game is running, and calls label refreshes via SwingUtilities.invokeLater(). Rounding reset sets the counter and label to zero. The thread starts at the initialization of GamePanel. It is not yet stopped application switching to another window, so proper management is required.

6.3.2 Bot Calculation Thread

GameController.executeAsynchronousBotThread() launches a daemon thread, sleeps for 750 milliseconds for simulation of thinking time, calculates the SmartBot move while synchronizing on GameModel, and all Swing updates get scheduled on the Event Dispatch Thread. This ensures that the click event is not blocked due to artificial delay or bot search.

```
                gameBoard.updateCell(row, col, processingMark);
                lblMovesTracked.setText(String.valueOf(gameModel.getMoveCount()));

                int[][] winningLine = gameModel.checkWinCoordinates();
                if (winningLine != null) {
                    gameModel.setGameOver(true);
                    gameBoard.highlightWinningCells(winningLine);

                    if (processingMark.equals(gameModel.getPlayer1Symbol())) {
                        gameModel.incrementHumanWins();
                        lblStatusText.setText("You won!");
                        lblHumanScoreboard.setText(String.valueOf(gameModel.getHumanWins()));
                    } else {
                        gameModel.incrementBotWins();
                        lblStatusText.setText(gameModel.getGameMode().equals("Singleplayer") ? "You lost!" : "Player 2 Wins!");
                        lblBotScoreboard.setText(String.valueOf(gameModel.getBotWins()));
                    }
                    // Notify game-over callback
                    if (gameOverCallback != null) {
                        SwingUtilities.invokeLater(gameOverCallback);
                    }
                    return;
                }

                if (gameModel.checkDraw()) {
                    gameModel.setGameOver(true);
                    gameModel.incrementDraws();
                    lblStatusText.setText("Draw!");
                    // Notify game-over callback
                    if (gameOverCallback != null) {
                        SwingUtilities.invokeLater(gameOverCallback);
                    }
                    return;
                }

                executionMode.processTurnTransition(gameModel);
                updateInterfaceDisplay();
            }
        } catch (IndexOutOfBoundsException e) {
            System.err.println("Encountered illegal grid assignment indexing error bounds coordinate trace: " + e.getMessage());
        }
    }

    private void executeAsynchronousBotThread() {
        Thread backgroundBotThread = new Thread(() -> {
            try {
                Thread.sleep(750);
            } catch (InterruptedException ex) {
                System.err.println("Asynchronous computing thread operation halted unexpectedly mid-execution.");
                Thread.currentThread().interrupt();
                return;
            }

            Point selectedTarget = null;
            try {
                synchronized (gameModel) {
                    selectedTarget = opponentIntelligence.determineOptimalMove(gameModel);
                }
            } catch (NullPointerException | ArrayIndexOutOfBoundsException ex) {
                System.err.println("Intelligence algorithm model execution dropped tracking context variables: " + ex.getMessage());
            }

            final Point finalTarget = selectedTarget;
            SwingUtilities.invokeLater(() -> {
                try {
                    synchronized (gameModel) {
                        String[][] currentMatrix = gameModel.getBoardState();
                        int gridLength = gameModel.getBoardSize();
                        gameBoard.initializeBoard();
                        for (int r = 0; r < gridLength; r++) {
                            for (int c = 0; c < gridLength; c++) {
                                if (!"".equals(currentMatrix[r][c])) {
                                    gameBoard.updateCell(r, c, currentMatrix[r][c]);
                                }
                            }
                        }
                    }
                    if (finalTarget != null) {
                        processTurnMove(finalTarget.x, finalTarget.y);
                    }
                } catch (Exception ex) {
                    System.err.println("Failed syncing background thread calculation state to UI workspace canvas: " + ex.getMessage());
                }
            });
        });

```

Figure 6.3: GameController.java asynchronous bot calculation and EDT update.

6.3.3 Background Music Thread

GamePanel creates an object of MusicPlayer and passes it to the thread that is working as a daemon. MusicPlayer implements a Runnable interface and plays the WAV stream in run() method. Clip keeps on looping infinitely. Errors related to audio are caught in the worker itself.

```java
package main.ui;

import javax.sound.sampled.*;
import java.io.File;
import java.io.IOException;
import java.net.URL;
import main.exceptions.GameException;

public class MusicPlayer implements Runnable {
    private Clip audioClip;
    private boolean isMuted = false;
    private final String fallbackFilePath;

    public MusicPlayer(String fallbackFilePath) {
        this.fallbackFilePath = fallbackFilePath;
    }

    @Override
    public void run() {
        try {
            openAudioStream();
        } catch (GameException e) {
            System.err.println(e.toString());
            System.err.println("Audio processing system dropped safely into silent execution mode.");
        }
    }

    private void openAudioStream() throws GameException {
        AudioInputStream stream = null;
        try {
            File localFile = new File(fallbackFilePath);
            if (localFile.exists()) {
                stream = AudioSystem.getAudioInputStream(localFile);
            } else {
                URL resourceUrl = getClass().getResource("/resources/sounds/background.wav");
                if (resourceUrl != null) {
                    stream = AudioSystem.getAudioInputStream(resourceUrl);
                }
            }

            if (stream == null) {
                throw new GameException("Target background theme soundtrack audio resource file missing entirely.", GameException.ErrorSeverity.INFO);
            }

            audioClip = AudioSystem.getClip();
            audioClip.open(stream);
            audioClip.loop(Clip.LOOP_CONTINUOUSLY);

            synchronized (this) {
                if (isMuted) {
                    applyMuteState(true);
                }
            }
        } catch (UnsupportedAudioFileException e) {
            throw new GameException("Audio stream initialization failed: File format parameters are incompatible.", e, GameException.ErrorSeverity.WARNING);
        } catch (LineUnavailableException e) {
            throw new GameException("System sound architecture allocation channel rejected stream request.", e, GameException.ErrorSeverity.WARNING);
        } catch (IOException e) {
            throw new GameException("Low-level hardware disk interface failure reading sound stream block.", e, GameException.ErrorSeverity.WARNING);
        } finally {
            // Guarantee stream closing process to clear out operating system locks
            if (stream != null) {
                try {
                    stream.close();
                } catch (IOException e) {
                    System.err.println("Warning: Resource management block leaked closing audio stream: " + e.getMessage());
                }
            }
        }
    }

    public synchronized void toggleMute() {
        isMuted = !isMuted;
        if (audioClip != null) {
            applyMuteState(isMuted);
        }
    }

    private void applyMuteState(boolean mute) {
        try {
            if (audioClip.isControlSupported(BooleanControl.Type.MUTE)) {
                BooleanControl control = (BooleanControl) audioClip.getControl(BooleanControl.Type.MUTE);
                control.setValue(mute);
            } else if (audioClip.isControlSupported(FloatControl.Type.MASTER_GAIN)) {
                FloatControl gainControl = (FloatControl) audioClip.getControl(FloatControl.Type.MASTER_GAIN);
                gainControl.setValue(mute ? gainControl.getMinimum() : 0.0f);
            }
        } catch (IllegalArgumentException e) {
            System.err.println("Target audio master controllers rejected dynamic sound balance adjustments: " + e.getMessage());
        }
    }
```

Figure 6.4: MusicPlayer.java background audio and controlled exception handling.

6.4 Exception Handling

6.4.1 Database Exceptions

DatabaseConnection throws ClassNotFoundException and SQLException in terms of GameException with a specified severity. Likewise, SettingsDAO handles SQLExceptions in that way. In Main and SettingsPanel classes, GameException is caught, memory default or runtime settings are kept, and warnings are shown. Unused DatabaseException class can be safely deleted or uniformly used.

6.4.2 Audio Exceptions

Any unsupported audio format, unavailable sound line, or I/O error is translated into GameException by MusicPlayer. run() method catches this exception and turns silent. The stream is closed within finally block to minimize resource leakages..

6.4.3 Interface and Game Logic Exceptions

Catches for main are related to mistakes in interface construction, while GameController catches are related to mistakes in indexing and clicking processing. Board updates validate coordinates. InterruptedException restores interrupted state of worker threads. There is catching of general Exception or NullPointerException, which should be fixed with better validations in the future.

6.5 JDBC Database Connectivity

JDBC offers a Java API that enables making connections, running SQL queries, and processing data [2]. The project relies on DriverManager.getConnection() with jdbc:mysql://localhost:3306/tic_tac_toe_db connection URL and loading of com.mysql.cj.jdbc.Driver explicitly. SettingsDAO makes use of parameterized query instead of the String concatenation of SQL. try-with-resources closes JDBC resources automatically. At runtime, MySQL Connector/J and local MySQL server with provided schema have to be used.

```java
package main.database;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import main.game.GameModel;
import main.exceptions.GameException;

public class SettingsDAO {

    /**
     * Persist all settings from the GameModel into the database.
     * Expects a table 'game_settings' with columns:
     * id (INT PK), gamemode (VARCHAR), board_size (INT),
     * match_timer (BOOLEAN), board_info (BOOLEAN), player_counter (BOOLEAN).
     */
    public static void saveSettings(GameModel model) throws GameException {
        String query = "UPDATE game_settings SET gamemode = ?, board_size = ?, match_timer = ?, board_info = ?, player_counter = ? WHERE id = 1";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(query)) {

            ps.setString(1, model.getGameMode());
            ps.setInt(2, model.getBoardSize());
            ps.setBoolean(3, model.isMatchTimerEnabled());
            ps.setBoolean(4, model.isBoardInfoEnabled());
            ps.setBoolean(5, model.isPlayerCounterEnabled());

            int rowsUpdated = ps.executeUpdate();
            if (rowsUpdated == 0) {
                // No row with id=1 → insert a new one
                String insertQuery = "INSERT INTO game_settings (id, gamemode, board_size, match_timer, board_info, player_counter) " +
                                     "VALUES (1, ?, ?, ?, ?, ?)";
                try (PreparedStatement psInsert = conn.prepareStatement(insertQuery)) {
                    psInsert.setString(1, model.getGameMode());
                    psInsert.setInt(2, model.getBoardSize());
                    psInsert.setBoolean(3, model.isMatchTimerEnabled());
                    psInsert.setBoolean(4, model.isBoardInfoEnabled());
                    psInsert.setBoolean(5, model.isPlayerCounterEnabled());
                    psInsert.executeUpdate();
                }
            }
        } catch (SQLException e) {
            throw new GameException("Persistent layer write operation halted writing settings.", e, GameException.ErrorSeverity.WARNING);
        }
    }

    /**
     * Load all settings from the database into the given GameModel.
     * If no row exists, default values remain unchanged (they are set to true in the model).
     */
    public static void loadSettings(GameModel model) throws GameException {
        String query = "SELECT gamemode, board_size, match_timer, board_info, player_counter FROM game_settings WHERE id = 1";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(query);
             ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {
                model.setGameMode(rs.getString("gamemode"));
                model.setBoardSize(rs.getInt("board_size"));
                model.setMatchTimerEnabled(rs.getBoolean("match_timer"));
                model.setBoardInfoEnabled(rs.getBoolean("board_info"));
                model.setPlayerCounterEnabled(rs.getBoolean("player_counter"));
            } else {
                // No configuration found; keep model defaults (true) and warn
                throw new GameException("Initial configuration record missing from database. Using default settings.", GameException.ErrorSeverity.INFO);
            }
        } catch (SQLException e) {
            throw new GameException("Configuration extraction sequence dropped out. Defaulting to local memory fallback settings.", e, GameException.ErrorSeverity.INFO);
        }
    }
```

Figure 6.5: SettingsDAO.java prepared statements and try-with-resources.

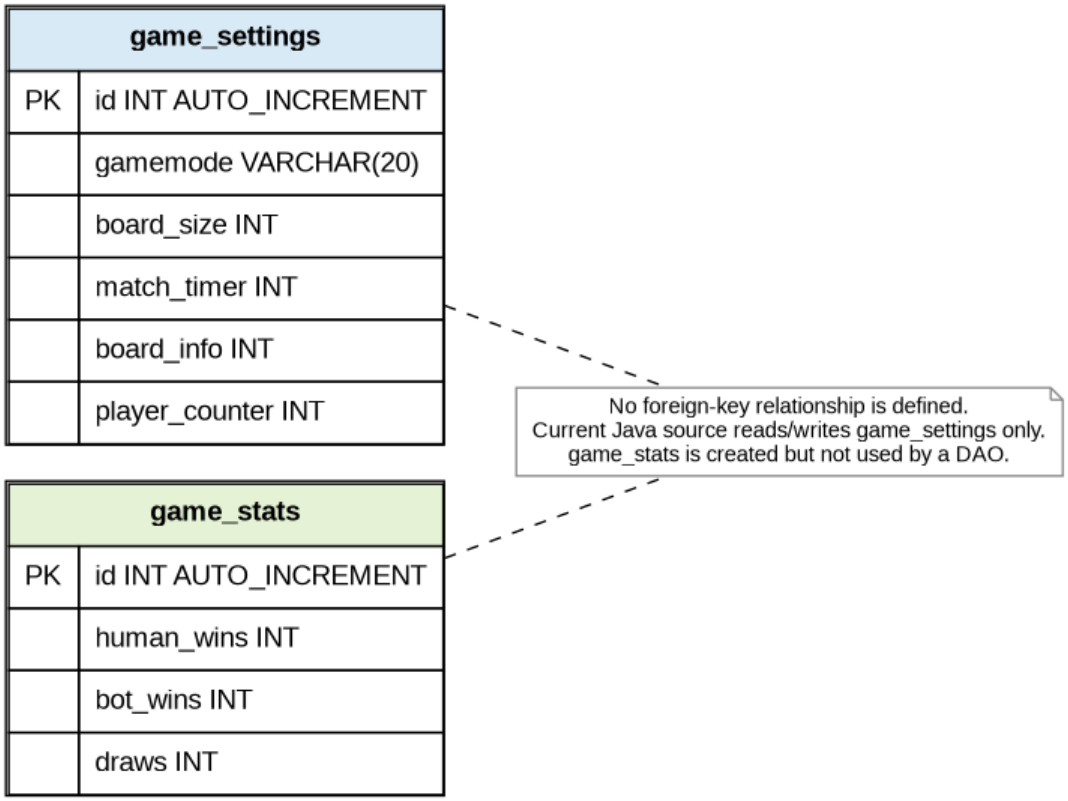


Figure 6.6: Database tables defined by database/schema.sql.

6.6 Screenshot of Output: Welcome Screen

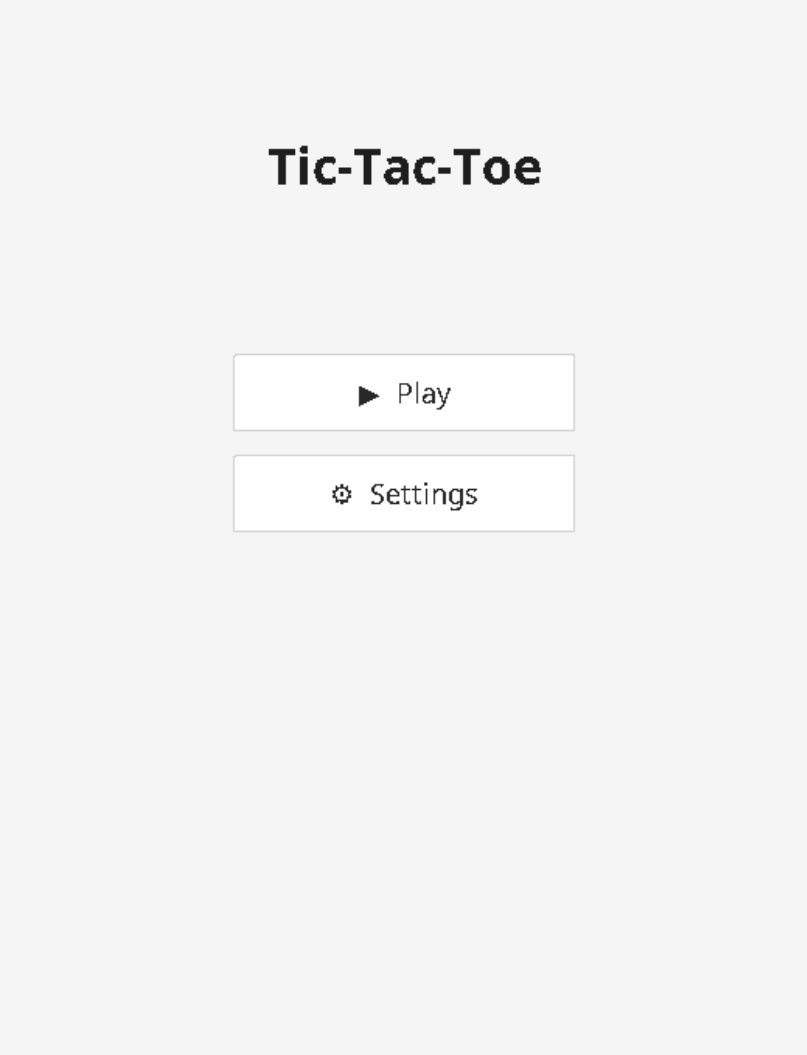


Figure 6.7: Welcome screen with Play and Settings buttons.

The welcome screen matches the submitted WelcomePanel: a centred title and two navigation buttons on a light-grey background.

6.7 Screenshot of Output: Gameplay Screen

6.7.1 Standard 3x3 Board

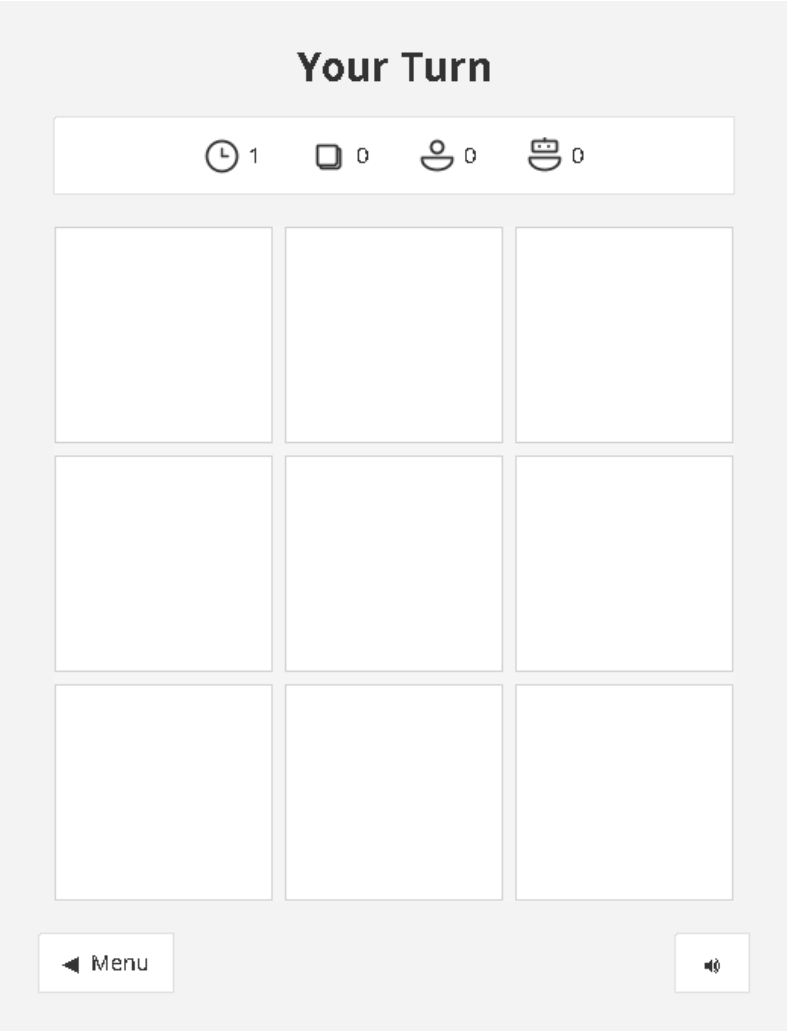


Figure 6.8: Default 3x3 gameplay board.

6.7.2 Custom 4x4, 5x5, and 6x6 Boards

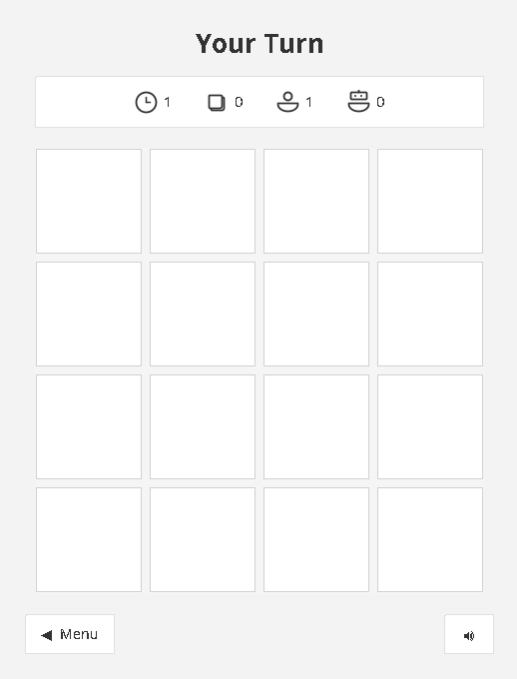


Figure 6.9(a): 4x4 board.

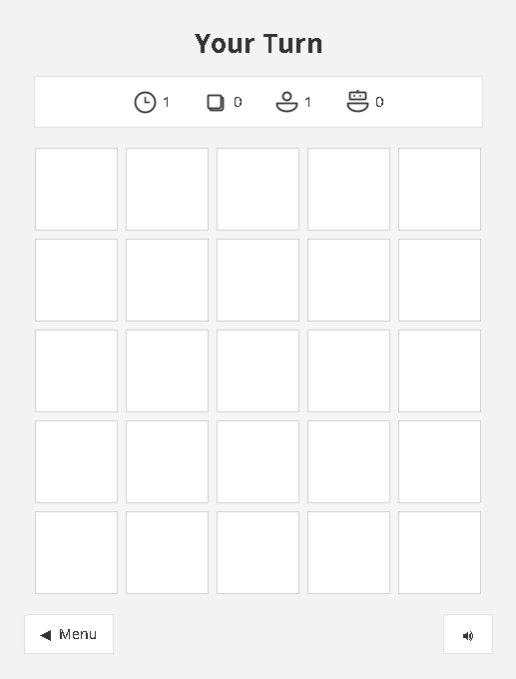


Figure 6.9(b): 5x5 board.

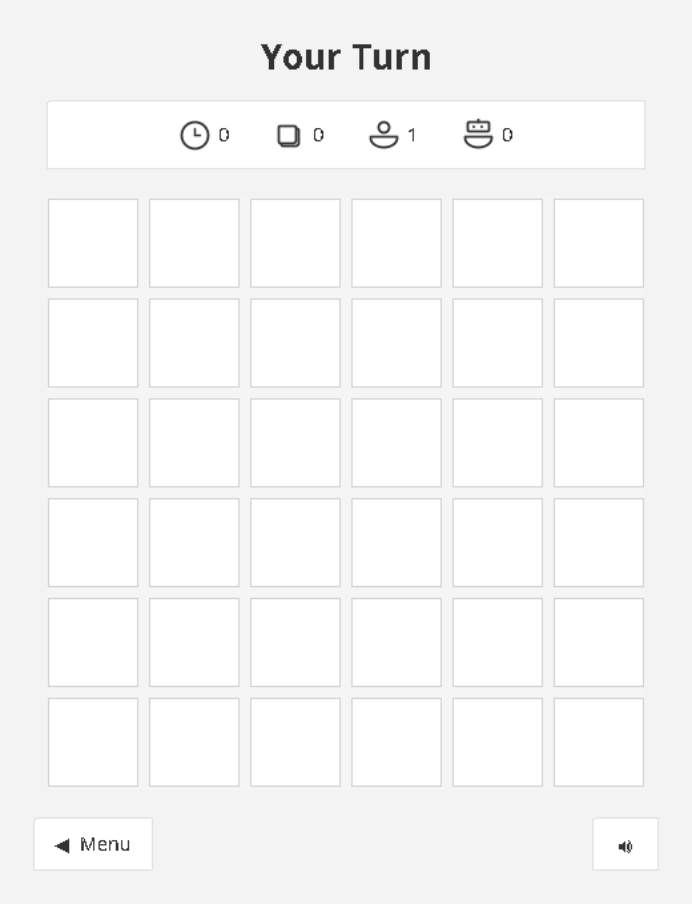


Figure 6.9(c): 6x6 board.

The same GameBoard component is rebuilt for each selected dimension. Cell spacing and the enclosing layout remain consistent.

6.7.3 Singleplayer Gameplay

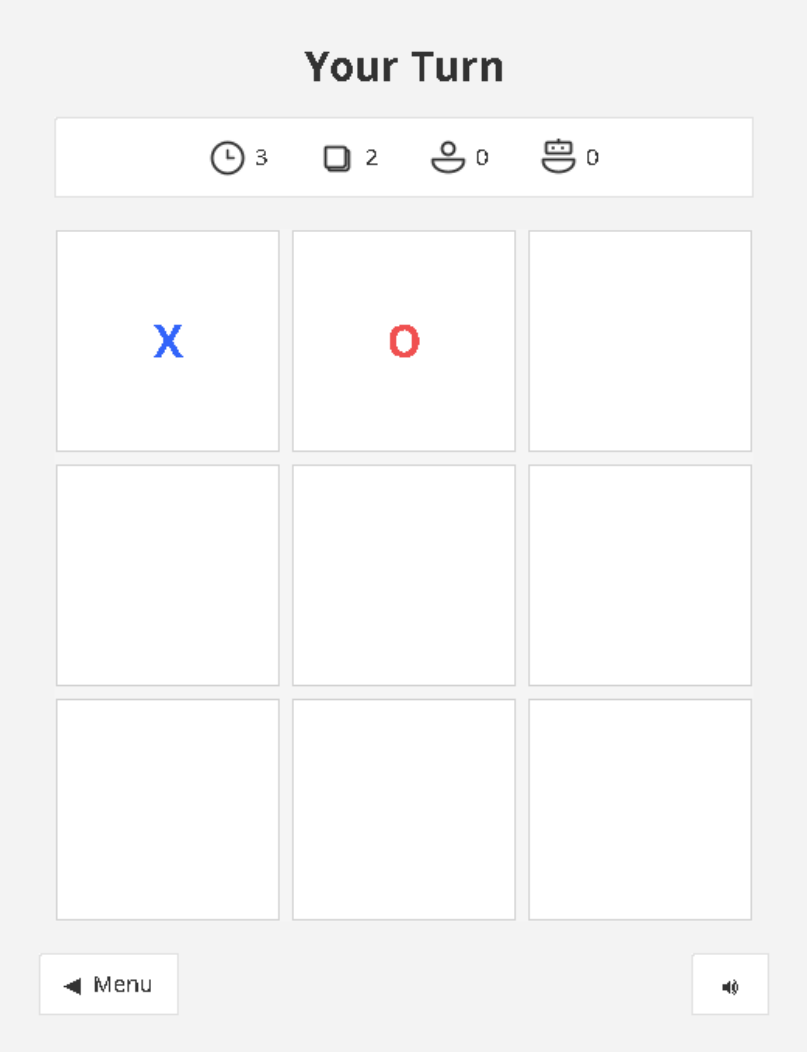


Figure 6.10: Human X move followed by an asynchronous bot O move.

6.7.4 Multiplayer Gameplay

Multiplayer uses the same board and symbols but alternates status between Player 1 and Player 2 without invoking SmartBot. Figure 6.11 also demonstrates a completed multiplayer sequence.

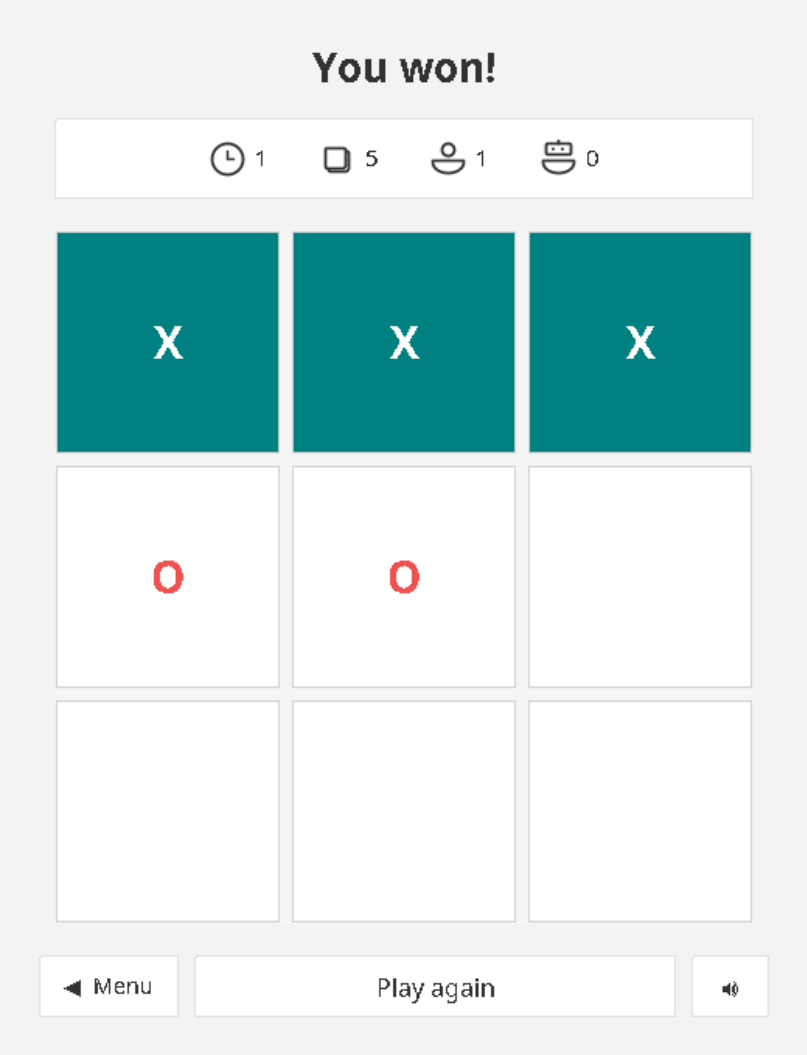


Figure 6.11: Multiplayer win after alternating X and O turns.

6.7.5 Winning-Line Highlight

The completed top row is highlighted in teal. The coordinates originate from GameModel.checkWinCoordinates() and are applied by GameBoard.highlightWinningCells().

6.7.6 Draw Result

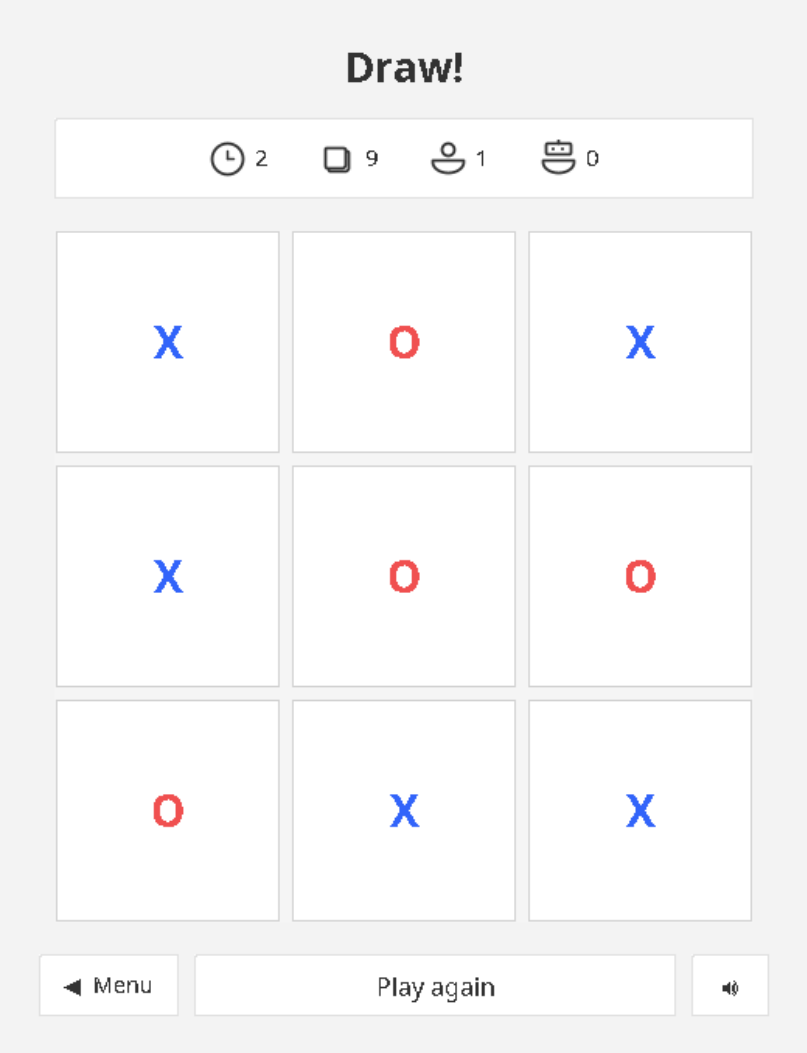


Figure 6.12: Draw result after all nine cells are occupied without a winner.

6.7.7 Mute and Unmute Function

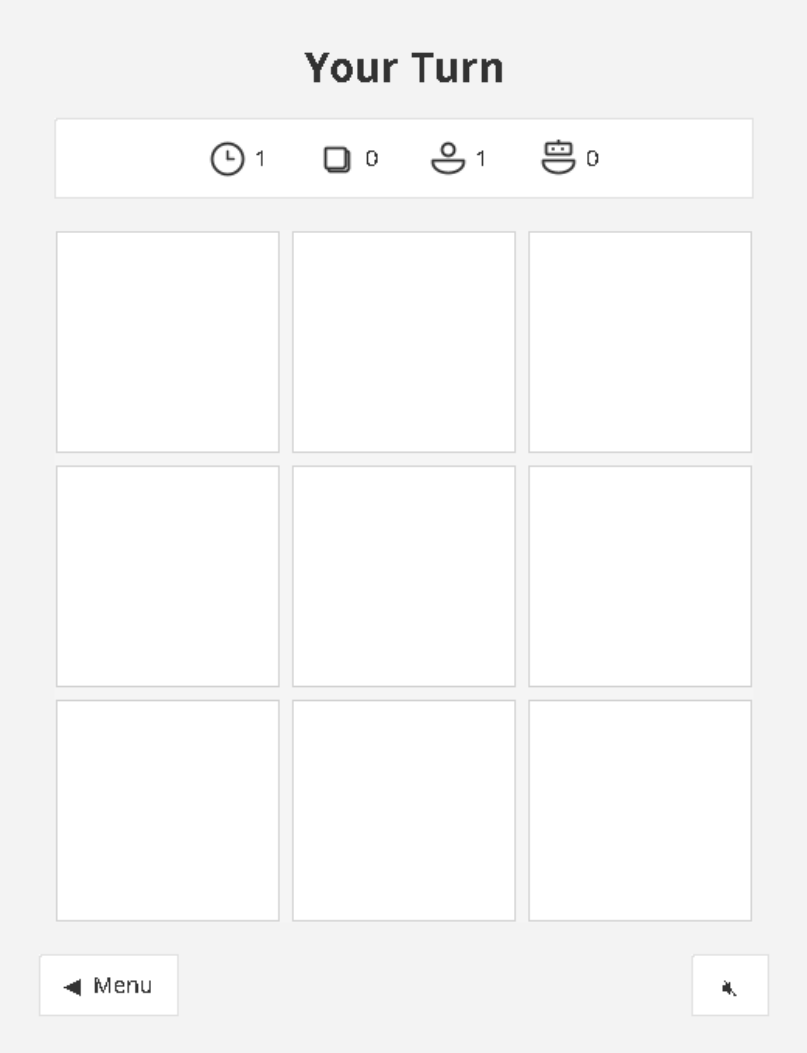


Figure 6.13: Gameplay screen after the mute control is toggled.

The bottom-right control changes state after calling MusicPlayer.toggleMute(). Audible playback itself was not independently verified in the headless test environment, but the WAV file structure and mute control logic were verified.

6.8 Screenshot of Output: Settings Screen

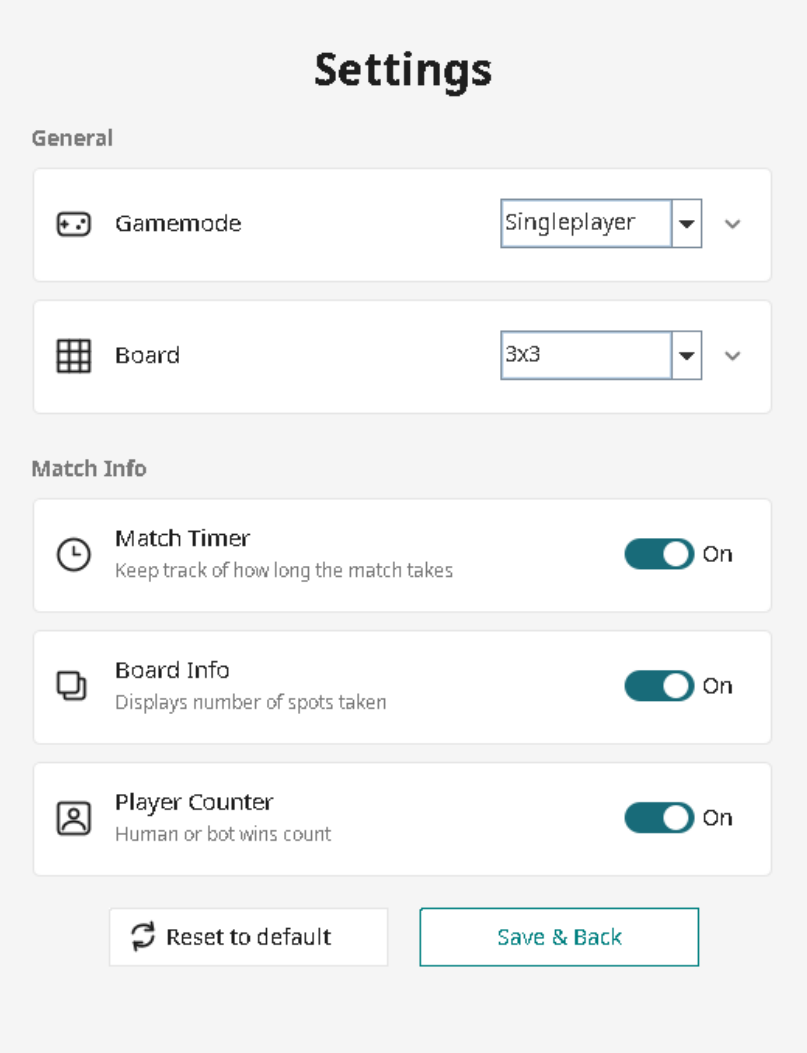


Figure 6.14: Settings screen with mode, board size, and match-information controls.

The screen exposes exactly two modes, four board-size choices, three switches, a reset action, and a save-and-return action.

6.9 Screenshot of Output: MySQL Database

```sql
CREATE DATABASE IF NOT EXISTS tic_tac_toe_db;
USE tic_tac_toe_db;

-- Table to store user configurations persistently
CREATE TABLE IF NOT EXISTS game_settings (
    id INT PRIMARY KEY AUTO_INCREMENT,
    gamemode VARCHAR(20) DEFAULT 'Singleplayer',
    board_size INT DEFAULT 3,
    match_timer INT DEFAULT 1,
    board_info INT DEFAULT 1,
    player_counter INT DEFAULT 1
);

-- Table to store performance statistics
CREATE TABLE IF NOT EXISTS game_stats (
    id INT PRIMARY KEY AUTO_INCREMENT,
    human_wins INT DEFAULT 0,
    bot_wins INT DEFAULT 0,
    draws INT DEFAULT 0
);

-- Seed initial records if they do not exist to prevent query errors
INSERT INTO game_settings (id, gamemode, board_size, match_timer, board_info, player_counter)
SELECT 1, 'Singleplayer', 3, 1, 1, 1 FROM DUAL
WHERE NOT EXISTS (SELECT 1 FROM game_settings WHERE id = 1);

INSERT INTO game_stats (id, human_wins, bot_wins, draws)
SELECT 1, 0, 0, 0 FROM DUAL
WHERE NOT EXISTS (SELECT 1 FROM game_stats WHERE id = 1);
```

Figure 6.15: Verifiable SQL schema supplied with the project.

7.0 SYSTEM TESTING

Combined compilation, automated model and strategy testing, and captured user interface testing. This project compiled successfully using javac 21. The full automated test results table is found in Appendix C, where there were 29 cases; 28 passed while one revealed a limitation at the model level validation. Database and audiable audio testings were done separately due to lack of environment..

7.1 Welcome-Screen Navigation Test

Result	Evidence
PASS	Play and Settings listeners are present and both panels were captured from the compiled UI.

7.2 Valid and Invalid Move Test

Result	Evidence
PASS	Valid empty cells were accepted; duplicate and out-of-range moves were rejected.

7.3 Singleplayer Bot Test

Result	Evidence
PASS	SmartBot selected an immediate winning move and blocked an immediate human win; UI capture showed bot response.

7.4 Multiplayer Turn Test

Result	Evidence
PASS	Mode status changed from Player 1 Turn to Player 2 Turn after a successful move.

7.5 Win Detection Test

Result	Evidence
PASS	Rows, columns, main diagonals, and anti-diagonals passed for 3x3, 4x4, 5x5, and 6x6.

7.6 Draw Detection Test

Result	Evidence

PASS	A full 3x3 board without a winning line returned draw.

7.7 Board-Size Test

Result	Evidence
PASS WITH LIMITATION	UI offered and rendered 3x3 to 6x6. Direct model assignment accepted 7, so validation belongs only to the current UI.

7.8 Audio Test

Result	Evidence
PARTIALLY VERIFIED	The WAV file is valid and mute logic is present. Audible output was not confirmed in the headless environment.

7.9 Timer Test

Result	Evidence
PASS	Timer label increased in compiled UI captures and updates are scheduled through SwingUtilities.invokeLater().

7.10 Database Save and Load Test

Result	Evidence
NOT CONFIRMED AT RUNTIME	SQL, JDBC code, prepared statements, and fallback handling were reviewed; connector JAR and MySQL service were absent.

7.11 Exception-Handling Test

Result	Evidence
PASS	Missing driver and missing audio resource paths were handled without terminating the interface in verification runs.

8.0 RESULTS AND CONCLUSION

The submitted project successfully demonstrates a working Java Swing Tic-Tac-Toe interface with CardLayout navigation, dynamic 3x3 to 6x6 grids, singleplayer and multiplayer turn handling, complete-line win detection, draw detection, score counters, replay behaviour, match-information visibility, a timer thread, background-music control, custom exceptions, and JDBC settings code. The source compiled successfully and the core model, mode, and bot logic passed 28 automated cases.

The report also identifies implementation boundaries that must not be overstated. SmartBot is a win/block/random heuristic and no Minimax or selectable difficulty feature exists. game_stats is not accessed by Java code. Model-level board-size validation is absent. The database connection was not independently executed because the ZIP refers to an external Connector/J path and no MySQL server was supplied. Therefore, the strongest verified conclusion is that the interface and core game logic are operational, while final database proof must be added from the group's configured local environment.

Overall, the project applies the principal Visual Programming topics required by the course: graphical components, event handling, object-oriented organisation, multithreading, exception handling, audio processing, and database connectivity code. Completing the environment-dependent database evidence and the improvements below would make the submission more portable and robust.

9.0 FUTURE IMPROVEMENTS

- Bundle dependencies through Maven or Gradle instead of an absolute Eclipse classpath entry, and document the exact Connector/J version and launch command.
- Validate board size inside GameModel.setBoardSize() so values outside 3 to 6 are rejected even when the model is called outside SettingsPanel.
- Implement a GameStatsDAO or remove game_stats. Current scores exist only in memory despite the table being created.
- Replace raw Thread creation with SwingWorker or an ExecutorService where appropriate, and stop timer and audio resources when the frame closes or the gameplay panel is no longer active.
- Use consistent typed exceptions and remove or integrate the currently unused DatabaseException class.
- Add player-name labels for multiplayer and replace the human/robot score icons according to the selected mode.
- Add deterministic unit tests, automated Swing UI tests, and a database integration test that runs against a temporary test schema.
- Improve accessibility with keyboard navigation, focus indicators, tooltips, accessible names, and optional high-contrast or scalable layouts.
- Add optional bot strategies or difficulty selection only after implementing and testing them; do not label the current heuristic as Minimax.

10.0 REFERENCES

[1] Oracle. “Concurrency in Swing” and “The Event Dispatch Thread.” Java Tutorials. https://docs.oracle.com/javase/tutorial/uiswing/concurrency/

[2] Oracle. “JDBC Basics.” Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/

[3] Oracle. “Lesson: Concurrency.” Java Tutorials. https://docs.oracle.com/javase/tutorial/essential/concurrency/

[4] Oracle. Java Platform API documentation for javax.sound.sampled. https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/sound/sampled/package-summary.html

[5] Oracle MySQL. “MySQL Connector/J Developer Guide.” https://dev.mysql.com/doc/connector-j/en/

[6] Group 13. vp-tic-tac-toe-main.zip: submitted Java source code, SQL schema, WAV resource, and Eclipse configuration, 2026.

[7] Group 13. progressprojectvpg13.pdf: project progress report, BIE20404 Visual Programming, 2026.

[8] Group 3. group3_bie20404.pdf: report layout and section-presentation reference only, BIE20404 Visual Programming, 2026.

APPENDICES

Appendix A: Database SQL Script

The following script is reproduced from database/schema.sql without changing its database name, tables, columns, defaults, or seed statements.

```
CREATE DATABASE IF NOT EXISTS tic_tac_toe_db;
USE tic_tac_toe_db;

-- Table to store user configurations persistently
CREATE TABLE IF NOT EXISTS game_settings (
    id INT PRIMARY KEY AUTO_INCREMENT,
    gamemode VARCHAR(20) DEFAULT 'Singleplayer',
    board_size INT DEFAULT 3,
    match_timer INT DEFAULT 1,
    board_info INT DEFAULT 1,
    player_counter INT DEFAULT 1
);

-- Table to store performance statistics
CREATE TABLE IF NOT EXISTS game_stats (
    id INT PRIMARY KEY AUTO_INCREMENT,
    human_wins INT DEFAULT 0,
    bot_wins INT DEFAULT 0,
    draws INT DEFAULT 0
);

-- Seed initial records if they do not exist to prevent query errors
INSERT INTO game_settings (id, gamemode, board_size, match_timer, board_info, player_counter)
SELECT 1, 'Singleplayer', 3, 1, 1, 1 FROM DUAL
WHERE NOT EXISTS (SELECT 1 FROM game_settings WHERE id = 1);

INSERT INTO game_stats (id, human_wins, bot_wins, draws)
SELECT 1, 0, 0, 0 FROM DUAL
WHERE NOT EXISTS (SELECT 1 FROM game_stats WHERE id = 1);
```

Appendix B: Important Source-Code Extracts

```java
package main;

import javax.swing.*;
import java.awt.*;
import main.ui.WelcomePanel;
import main.ui.GamePanel;
import main.ui.SettingsPanel;
import main.game.GameModel;
import main.database.DatabaseConnection;
import main.database.SettingsDAO;
import main.exceptions.GameException;

public class Main extends JFrame {
    private CardLayout cardLayout;
    private JPanel mainPanel;
    private GameModel model;
    private GamePanel gamePanel;
    private SettingsPanel settingsPanel;

    public Main() {
        setTitle("Tic-Tac-Toe");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(520, 680);
        setLocationRelativeTo(null);
        setResizable(false);

        cardLayout = new CardLayout();
        mainPanel = new JPanel(cardLayout);
        mainPanel.setBackground(new Color(245, 245, 245));

        model = new GameModel();
        loadSettingsIntoModel();   // Load saved settings before building the UI

        welcomePanelInitialized();

        add(mainPanel);
        cardLayout.show(mainPanel, "WelcomePanel");
    }

    // ------ Load settings from database and apply to model (fixes bugs 2, 4, 5) ------
    private void loadSettingsIntoModel() {
        try {
            // This static method reads the database and updates the given model directly.
            SettingsDAO.loadSettings(model);
        } catch (GameException e) {
            // If database is offline or settings not yet saved, keep defaults (3x3, Singleplayer, all visible)
            System.err.println("Could not load settings: " + e.getMessage());
            // Optionally log the stack trace for debugging
            // e.printStackTrace();
        }
    }

    private void welcomePanelInitialized() {
        try {
            WelcomePanel welcomePanel = new WelcomePanel(this);
            gamePanel = new GamePanel(this, model);
            settingsPanel = new SettingsPanel(this, model);

            mainPanel.add(welcomePanel, "WelcomePanel");
            mainPanel.add(gamePanel, "GamePanel");
            mainPanel.add(settingsPanel, "SettingsPanel");
        } catch (NullPointerException e) {
            System.err.println("Fatal Interface Generation Breakdown: Render construction stack collapsed: " + e.getMessage());
            JOptionPane.showMessageDialog(null,
                "A critical interface render framework exception occurred. Shutting down.",
                "Fatal System Failure",
                JOptionPane.ERROR_MESSAGE);
            System.exit(1);
        }
    }

    public void switchToPanel(String panelName) {
        try {
            cardLayout.show(mainPanel, panelName);
        } catch (IllegalArgumentException e) {
            System.err.println("Attempted invalid CardLayout component redirection to tag identity: " + panelName);
        }
    }

    public GamePanel getGamePanel() {
        return gamePanel;
    }

    public static void main(String[] args) {
        try {
            DatabaseConnection.testConnection();
        } catch (GameException e) {
            System.err.println(e.toString());
            JOptionPane.showMessageDialog(null,
                "MySQL service appears offline on local environment.\nApplication features will operate in safe local memory state.",
                "Database Environment Offline",
                JOptionPane.WARNING_MESSAGE);
        }

        SwingUtilities.invokeLater(() -> {
            try {
                Main frame = new Main();
                frame.setVisible(true);
            } catch (Exception e) {
                System.err.println("Fatal structural failure inside Event Dispatch Thread instantiation frame loop: " + e.getMessage());
            }
        });
    }
```

Appendix Figure B.1: Main.java navigation and application startup.

```java
        this.boardSize = 3; // Default visual grid setting
        this.gameMode = "Singleplayer";
        this.player1Symbol = "X";
        this.player2Symbol = "O";
        resetGame();
    }

    public synchronized void resetGame() {
        this.board = new String[boardSize][boardSize];
        for (int i = 0; i < boardSize; i++) {
            for (int j = 0; j < boardSize; j++) {
                board[i][j] = "";
            }
        }
        this.currentPlayer = player1Symbol;
        this.moveCount = 0;
        this.gameOver = false;
    }

    public synchronized boolean setMove(int row, int col) {
        if (row < 0 || row >= boardSize || col < 0 || col >= boardSize || !board[row][col].equals("") || gameOver) {
            return false;
        }
        board[row][col] = currentPlayer;
        moveCount++;
        return true;
    }

    public synchronized void switchPlayer() {
        if (currentPlayer.equals(player1Symbol)) {
            currentPlayer = player2Symbol;
        } else {
            currentPlayer = player1Symbol;
        }
    }

    // Dynamic NxN Win Detection Engine returning exact grid coordinates for Teal highlighting
    public synchronized int[][] checkWinCoordinates() {
        // 1. Check Rows
        for (int r = 0; r < boardSize; r++) {
            boolean win = true;
            String symbol = board[r][0];
            if (!symbol.equals("")) {
                for (int c = 1; c < boardSize; c++) {
                    if (!board[r][c].equals(symbol)) {
                        win = false;
                        break;
                    }
                }
                if (win) {
                    int[][] coords = new int[boardSize][2];
                    for (int c = 0; c < boardSize; c++) {
                        coords[c][0] = r;
                        coords[c][1] = c;
                    }
                    return coords;
                }
            }
        }

        // 2. Check Columns
        for (int c = 0; c < boardSize; c++) {
            boolean win = true;
            String symbol = board[0][c];
            if (!symbol.equals("")) {
                for (int r = 1; r < boardSize; r++) {
                    if (!board[r][c].equals(symbol)) {
                        win = false;
                        break;
                    }
                }
                if (win) {
                    int[][] coords = new int[boardSize][2];
                    for (int r = 0; r < boardSize; r++) {
                        coords[r][0] = r;
                        coords[r][1] = c;
                    }
                    return coords;
                }
            }
        }

        // 3. Check Primary Diagonal (Top-Left to Bottom-Right)
        String diag1Symbol = board[0][0];
        if (!diag1Symbol.equals("")) {
            boolean win = true;
            for (int i = 1; i < boardSize; i++) {
                if (!board[i][i].equals(diag1Symbol)) {
                    win = false;
                    break;
                }
            }
            if (win) {
                int[][] coords = new int[boardSize][2];
                for (int i = 0; i < boardSize; i++) {
                    coords[i][0] = i;
                    coords[i][1] = i;
                }
                return coords;
            }
        }

        // 4. Check Secondary Diagonal (Top-Right to Bottom-Left)
        String diag2Symbol = board[0][boardSize - 1];
        if (!diag2Symbol.equals("")) {
            boolean win = true;
            for (int i = 1; i < boardSize; i++) {
                if (!board[i][boardSize - 1 - i].equals(diag2Symbol)) {
                    win = false;
                    break;
                }
            }
            if (win) {
                int[][] coords = new int[boardSize][2];
                for (int i = 0; i < boardSize; i++) {
                    coords[i][0] = i;
                    coords[i][1] = boardSize - 1 - i;
                }
```

Appendix Figure B.2: GameModel.java state, move validation, and win detection.

```
                gameBoard.updateCell(row, col, processingMark);
                lblMovesTracked.setText(String.valueOf(gameModel.getMoveCount()));

                int[][] winningLine = gameModel.checkWinCoordinates();
                if (winningLine != null) {
                    gameModel.setGameOver(true);
                    gameBoard.highlightWinningCells(winningLine);

                    if (processingMark.equals(gameModel.getPlayer1Symbol())) {
                        gameModel.incrementHumanWins();
                        lblStatusText.setText("You won!");
                        lblHumanScoreboard.setText(String.valueOf(gameModel.getHumanWins()));
                    } else {
                        gameModel.incrementBotWins();
                        lblStatusText.setText(gameModel.getGameMode().equals("Singleplayer") ? "You lost!" : "Player 2 Wins!");
                        lblBotScoreboard.setText(String.valueOf(gameModel.getBotWins()));
                    }
                    // Notify game-over callback
                    if (gameOverCallback != null) {
                        SwingUtilities.invokeLater(gameOverCallback);
                    }
                    return;
                }

                if (gameModel.checkDraw()) {
                    gameModel.setGameOver(true);
                    gameModel.incrementDraws();
                    lblStatusText.setText("Draw!");
                    // Notify game-over callback
                    if (gameOverCallback != null) {
                        SwingUtilities.invokeLater(gameOverCallback);
                    }
                    return;
                }

                executionMode.processTurnTransition(gameModel);
                updateInterfaceDisplay();
            }
        } catch (IndexOutOfBoundsException e) {
            System.err.println("Encountered illegal grid assignment indexing error bounds coordinate trace: " + e.getMessage());
        }
    }

    private void executeAsynchronousBotThread() {
        Thread backgroundBotThread = new Thread(() -> {
            try {
                Thread.sleep(750);
            } catch (InterruptedException ex) {
                System.err.println("Asynchronous computing thread operation halted unexpectedly mid-execution.");
                Thread.currentThread().interrupt();
                return;
            }

            Point selectedTarget = null;
            try {
                synchronized (gameModel) {
                    selectedTarget = opponentIntelligence.determineOptimalMove(gameModel);
                }
            } catch (NullPointerException | ArrayIndexOutOfBoundsException ex) {
                System.err.println("Intelligence algorithm model execution dropped tracking context variables: " + ex.getMessage());
            }

            final Point finalTarget = selectedTarget;
            SwingUtilities.invokeLater(() -> {
                try {
                    synchronized (gameModel) {
                        String[][] currentMatrix = gameModel.getBoardState();
                        int gridLength = gameModel.getBoardSize();
                        gameBoard.initializeBoard();
                        for (int r = 0; r < gridLength; r++) {
                            for (int c = 0; c < gridLength; c++) {
                                if (!"".equals(currentMatrix[r][c])) {
                                    gameBoard.updateCell(r, c, currentMatrix[r][c]);
                                }
                            }
                        }
                    }
                    if (finalTarget != null) {
                        processTurnMove(finalTarget.x, finalTarget.y);
                    }
                } catch (Exception ex) {
                    System.err.println("Failed syncing background thread calculation state to UI workspace canvas: " + ex.getMessage());
                }
            });
        });

```

Appendix Figure B.3: GameController.java bot thread and EDT hand-off.

```java
package main.ui;

import javax.sound.sampled.*;
import java.io.File;
import java.io.IOException;
import java.net.URL;
import main.exceptions.GameException;

public class MusicPlayer implements Runnable {
    private Clip audioClip;
    private boolean isMuted = false;
    private final String fallbackFilePath;

    public MusicPlayer(String fallbackFilePath) {
        this.fallbackFilePath = fallbackFilePath;
    }

    @Override
    public void run() {
        try {
            openAudioStream();
        } catch (GameException e) {
            System.err.println(e.toString());
            System.err.println("Audio processing system dropped safely into silent execution mode.");
        }
    }

    private void openAudioStream() throws GameException {
        AudioInputStream stream = null;
        try {
            File localFile = new File(fallbackFilePath);
            if (localFile.exists()) {
                stream = AudioSystem.getAudioInputStream(localFile);
            } else {
                URL resourceUrl = getClass().getResource("/resources/sounds/background.wav");
                if (resourceUrl != null) {
                    stream = AudioSystem.getAudioInputStream(resourceUrl);
                }
            }

            if (stream == null) {
                throw new GameException("Target background theme soundtrack audio resource file missing entirely.", GameException.ErrorSeverity.INFO);
            }

            audioClip = AudioSystem.getClip();
            audioClip.open(stream);
            audioClip.loop(Clip.LOOP_CONTINUOUSLY);

            synchronized (this) {
                if (isMuted) {
                    applyMuteState(true);
                }
            }
        } catch (UnsupportedAudioFileException e) {
            throw new GameException("Audio stream initialization failed: File format parameters are incompatible.", e, GameException.ErrorSeverity.WARNING);
        } catch (LineUnavailableException e) {
            throw new GameException("System sound architecture allocation channel rejected stream request.", e, GameException.ErrorSeverity.WARNING);
        } catch (IOException e) {
            throw new GameException("Low-level hardware disk interface failure reading sound stream block.", e, GameException.ErrorSeverity.WARNING);
        } finally {
            // Guarantee stream closing process to clear out operating system locks
            if (stream != null) {
                try {
                    stream.close();
                } catch (IOException e) {
                    System.err.println("Warning: Resource management block leaked closing audio stream: " + e.getMessage());
                }
            }
        }
    }

    public synchronized void toggleMute() {
        isMuted = !isMuted;
        if (audioClip != null) {
            applyMuteState(isMuted);
        }
    }

    private void applyMuteState(boolean mute) {
        try {
            if (audioClip.isControlSupported(BooleanControl.Type.MUTE)) {
                BooleanControl control = (BooleanControl) audioClip.getControl(BooleanControl.Type.MUTE);
                control.setValue(mute);
            } else if (audioClip.isControlSupported(FloatControl.Type.MASTER_GAIN)) {
                FloatControl gainControl = (FloatControl) audioClip.getControl(FloatControl.Type.MASTER_GAIN);
                gainControl.setValue(mute ? gainControl.getMinimum() : 0.0f);
            }
        } catch (IllegalArgumentException e) {
            System.err.println("Target audio master controllers rejected dynamic sound balance adjustments: " + e.getMessage());
        }
    }
```

Appendix Figure B.4: MusicPlayer.java audio thread and exceptions.

```java
package main.database;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import main.game.GameModel;
import main.exceptions.GameException;

public class SettingsDAO {

    /**
     * Persist all settings from the GameModel into the database.
     * Expects a table 'game_settings' with columns:
     * id (INT PK), gamemode (VARCHAR), board_size (INT),
     * match_timer (BOOLEAN), board_info (BOOLEAN), player_counter (BOOLEAN).
     */
    public static void saveSettings(GameModel model) throws GameException {
        String query = "UPDATE game_settings SET gamemode = ?, board_size = ?, match_timer = ?, board_info = ?, player_counter = ? WHERE id = 1";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(query)) {

            ps.setString(1, model.getGameMode());
            ps.setInt(2, model.getBoardSize());
            ps.setBoolean(3, model.isMatchTimerEnabled());
            ps.setBoolean(4, model.isBoardInfoEnabled());
            ps.setBoolean(5, model.isPlayerCounterEnabled());

            int rowsUpdated = ps.executeUpdate();
            if (rowsUpdated == 0) {
                // No row with id=1 → insert a new one
                String insertQuery = "INSERT INTO game_settings (id, gamemode, board_size, match_timer, board_info, player_counter) " +
                                     "VALUES (1, ?, ?, ?, ?, ?)";
                try (PreparedStatement psInsert = conn.prepareStatement(insertQuery)) {
                    psInsert.setString(1, model.getGameMode());
                    psInsert.setInt(2, model.getBoardSize());
                    psInsert.setBoolean(3, model.isMatchTimerEnabled());
                    psInsert.setBoolean(4, model.isBoardInfoEnabled());
                    psInsert.setBoolean(5, model.isPlayerCounterEnabled());
                    psInsert.executeUpdate();
                }
            }
        } catch (SQLException e) {
            throw new GameException("Persistent layer write operation halted writing settings.", e, GameException.ErrorSeverity.WARNING);
        }
    }

    /**
     * Load all settings from the database into the given GameModel.
     * If no row exists, default values remain unchanged (they are set to true in the model).
     */
    public static void loadSettings(GameModel model) throws GameException {
        String query = "SELECT gamemode, board_size, match_timer, board_info, player_counter FROM game_settings WHERE id = 1";

        try (Connection conn = DatabaseConnection.getConnection();
             PreparedStatement ps = conn.prepareStatement(query);
             ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {
                model.setGameMode(rs.getString("gamemode"));
                model.setBoardSize(rs.getInt("board_size"));
                model.setMatchTimerEnabled(rs.getBoolean("match_timer"));
                model.setBoardInfoEnabled(rs.getBoolean("board_info"));
                model.setPlayerCounterEnabled(rs.getBoolean("player_counter"));
            } else {
                // No configuration found; keep model defaults (true) and warn
                throw new GameException("Initial configuration record missing from database. Using default settings.", GameException.ErrorSeverity.INFO);
            }
        } catch (SQLException e) {
            throw new GameException("Configuration extraction sequence dropped out. Defaulting to local memory fallback settings.", e, GameException.ErrorSeverity.INFO);
        }
    }
```

Appendix Figure B.5: SettingsDAO.java database operations.

```
300         resetBtn.setBorder(BorderFactory.createLineBorder(CARD_BORDER, 1, true));
301         resetBtn.setMargin(new Insets(0, 26, 0, 0));
302         resetBtn.addActionListener(e -> performDefaultReset());
303         return resetBtn;
304     }
305
306     private JButton createSaveButton() {
307         JButton saveBtn = new JButton("Save & Back");
308         saveBtn.setFont(new Font("Segoe UI", Font.PLAIN, 14));
309         saveBtn.setBackground(Color.WHITE);
310         saveBtn.setForeground(new Color(0, 128, 128));
311         saveBtn.setFocusPainted(false);
312         saveBtn.setPreferredSize(new Dimension(180, 38));
313         saveBtn.setBorder(BorderFactory.createLineBorder(new Color(0, 128, 128), 1, true));
314         saveBtn.addActionListener(e -> performPersistentSave());
315         return saveBtn;
316     }
317
318     // ---------- Backend Logic ----------
319
320     private void loadSettingsFromDatabase() {
321         try {
322             SettingsDAO.loadSettings(model);
323             // Gamemode
324             comboGamemode.setSelectedItem(model.getGameMode());
325             // Board size
326             int size = model.getBoardSize();
327             String sizeStr = size + "x" + size;
328             comboBoard.setSelectedItem(sizeStr);
329             // Toggles
330             toggleTimer.setActivated(model.isMatchTimerEnabled());
331             lblTimerState.setText(model.isMatchTimerEnabled() ? "On" : "Off");
332             toggleBoardInfo.setActivated(model.isBoardInfoEnabled());
333             lblBoardInfoState.setText(model.isBoardInfoEnabled() ? "On" : "Off");
334             toggleCounter.setActivated(model.isPlayerCounterEnabled());
335             lblCounterState.setText(model.isPlayerCounterEnabled() ? "On" : "Off");
336         } catch (GameException e) {
337             System.err.println(e.toString());
338             JOptionPane.showMessageDialog(this,
339                     "Database server connection is offline. Defaulting to safe offline memory structures.",
340                     "Persistence Layer Notice",
341                     JOptionPane.INFORMATION_MESSAGE);
342         }
343     }
344
345     private void performPersistentSave() {
346         // Read UI values
347         String selectedMode = (String) comboGamemode.getSelectedItem();
348         String sizeStr = (String) comboBoard.getSelectedItem();
349         int selectedSize = Integer.parseInt(sizeStr.substring(0, 1)); // "3x3" -> 3
350
351         model.setGameMode(selectedMode);
352         model.setBoardSize(selectedSize);
353         model.setMatchTimerEnabled(toggleTimer.isActivated());
354         model.setBoardInfoEnabled(toggleBoardInfo.isActivated());
355         model.setPlayerCounterEnabled(toggleCounter.isActivated());
356
357         try {
358             SettingsDAO.saveSettings(model);
359             JOptionPane.showMessageDialog(this,
360                     "Settings persistently synchronized inside DB schema configuration.",
361                     "Success", JOptionPane.INFORMATION_MESSAGE);
362         } catch (GameException e) {
363             System.err.println(e.toString());
364             JOptionPane.showMessageDialog(this,
365                     "SQL Server unavailable. Selection applied to runtime cache memory state successfully.",
366                     "Offline Memory Cache Mode Activated",
367                     JOptionPane.WARNING_MESSAGE);
368         } finally {
369             mainFrame.getGamePanel().synchronizeControllerSettings();
370             mainFrame.switchToPanel("WelcomePanel");
371         }
372     }
373
374     private void performDefaultReset() {
375         comboGamemode.setSelectedIndex(0);          // Singleplayer
376         comboBoard.setSelectedIndex(0);             // 3x3
377         toggleTimer.setActivated(true);
378         lblTimerState.setText("On");
379         toggleBoardInfo.setActivated(true);
380         lblBoardInfoState.setText("On");
381         toggleCounter.setActivated(true);
382         lblCounterState.setText("On");
383
384         model.setGameMode("Singleplayer");
385         model.setBoardSize(3);
386         model.setMatchTimerEnabled(true);
387         model.setBoardInfoEnabled(true);
388         model.setPlayerCounterEnabled(true);
389
390         try {
391             SettingsDAO.saveSettings(model);
392         } catch (GameException e) {
393             System.err.println("Warning: Local database save blocked on reset step. Cleaning memory registers locally: " + e.getMessage());
394         } finally {
395             mainFrame.getGamePanel().synchronizeControllerSettings();
396             JOptionPane.showMessageDialog(this,
397                     "Application runtime states reverted to defaults.",
398                     "Reset Complete", JOptionPane.INFORMATION_MESSAGE);
399         }
400     }
401
402     // ---------- Inner Classes: CustomIcon & CustomToggleSwitch (copied from TestGui) ----------
403
404     private class CustomIcon extends JComponent {
405         private final String type;
406
407         public CustomIcon(String type) {
408             this.type = type;
409             if (type.equals("chevron")) {
410                 setPreferredSize(new Dimension(12, 12));
```

Appendix Figure B.6: SettingsPanel.java save and reset operations.

Appendix C: Complete Test-Case Table

ID	Test Description	Result	Evidence
TC01	Default board size is 3x3	PASS	boardSize=3
TC02	Valid move accepted	PASS	setMove(0,0)
TC03	Occupied cell rejected	PASS	duplicate move rejected
TC04	Out-of-range move rejected	PASS	bounds checks
TC05	3x3 row win detected	PASS	row 1 filled with X
TC06	3x3 column win detected	PASS	column 1 filled with X
TC07	3x3 main diagonal win detected	PASS	main diagonal filled with X
TC08	3x3 anti-diagonal win detected	PASS	anti-diagonal filled with X
TC09	4x4 row win detected	PASS	row 1 filled with X
TC10	4x4 column win detected	PASS	column 1 filled with X
TC11	4x4 main diagonal win detected	PASS	main diagonal filled with X
TC12	4x4 anti-diagonal win detected	PASS	anti-diagonal filled with X
TC13	5x5 row win detected	PASS	row 1 filled with X
TC14	5x5 column win detected	PASS	column 1 filled with X
TC15	5x5 main diagonal win detected	PASS	main diagonal filled with X
TC16	5x5 anti-diagonal win detected	PASS	anti-diagonal filled with X
TC17	6x6 row win detected	PASS	row 1 filled with X
TC18	6x6 column win detected	PASS	column 1 filled with X
TC19	6x6 main diagonal win detected	PASS	main diagonal filled with X
TC20	6x6 anti-diagonal win detected	PASS	anti-diagonal filled with X
TC21	Full 3x3 board without winner is detected as draw	PASS	9 legal moves, no winning line
TC22	SmartBot chooses immediate winning move	PASS	selected=java.awt.Point[x=0,y=2]
TC23	SmartBot blocks immediate human win	PASS	selected=java.awt.Point[x=1,y=2]

ID	Test Description	Result	Evidence
TC24	RandomBot returns an empty legal cell	PASS	selected=java.awt.Point[x=0,y=3]
TC25	Singleplayer status begins with Your Turn	PASS	Your Turn
TC26	Singleplayer transition changes to Bot thinking	PASS	Bot thinking...
TC27	Multiplayer status begins with Player 1 Turn	PASS	Player 1 Turn
TC28	Multiplayer alternates to Player 2 Turn	PASS	Player 2 Turn